In [427…
```
#Due 3/20/26
#BSAN360-001 Final Project Code
#Luke Schrader
```

In [428…
```
#Lab 1
#Question 1 - importing pandas and reading the csv
import pandas as pd
pitcher = pd.read_csv('pitch type vs performance.csv')
```

In [429…
```
#Question 2 - printing the first 5 and last five rows, gives row x col
pitcher
#As displayed at the bottom of the data set, there are 809 rows and 52
#correct.
```

Out[429…

	last_name, first_name	player_id	year	p_game	p_formatted_ip	pa	ab	hit	str
0	Blanco, Ronel	669854	2025	9	48.1	199	176	37	
1	Kelly, Merrill	518876	2025	32	184.0	748	696	157	
2	Hamel, Dom	687589	2025	1	1.0	6	5	3	
3	Basso, Brady	669620	2025	11	11.2	56	47	12	
4	Montas, Frankie	593423	2025	9	38.2	178	161	48	
...	...	...	...	...	...	...	...	...	
804	Henriquez, Edgardo	683618	2025	22	19.0	78	72	17	
805	Peraza, Oswald	672724	2025	2	2.1	16	15	9	
806	Leclerc, José	600917	2025	10	9.0	46	41	13	
807	White, Owen	669391	2025	3	7.0	35	32	14	
808	Selby, Colin	681882	2025	11	14.0	60	58	16	

809 rows × 53 columns

In [430…
```
#Question 3 - First 5 rows, all looks normal
pitcher.head()
```

Out[430…

	last_name, first_name	player_id	year	p_game	p_formatted_ip	pa	ab	hit	strike
0	Blanco, Ronel	669854	2025	9	48.1	199	176	37	
1	Kelly, Merrill	518876	2025	32	184.0	748	696	157	
2	Hamel, Dom	687589	2025	1	1.0	6	5	3	
3	Basso, Brady	669620	2025	11	11.2	56	47	12	
4	Montas, Frankie	593423	2025	9	38.2	178	161	48	

5 rows × 53 columns

In [431…

```
#Question 3 - Last 5 rows, all looks normal
pitcher.tail()
```

Out[431…

	last_name, first_name	player_id	year	p_game	p_formatted_ip	pa	ab	hit	strike
804	Henriquez, Edgardo	683618	2025	22	19.0	78	72	17	
805	Peraza, Oswald	672724	2025	2	2.1	16	15	9	
806	Leclerc, José	600917	2025	10	9.0	46	41	13	
807	White, Owen	669391	2025	3	7.0	35	32	14	
808	Selby, Colin	681882	2025	11	14.0	60	58	16	

5 rows × 53 columns

In [432…

```
#Question 4 - Columns Renaming, printed before and after. Since player
#important to the data I am recording, I will keep the row index as is
#the player name column down the road.
print(pitcher.columns)
pitcher = pitcher.rename(columns={'last_name, first_name': 'Name','p_f
print(pitcher.columns)
pitcher
```

```
Index(['last_name, first_name', 'player_id', 'year', 'p_game',
       'p_formatted_ip', 'pa', 'ab', 'hit', 'strikeout', 'walk', 'k_per
cent',
       'bb_percent', 'batting_avg', 'slg_percent', 'on_base_percent',
       'on_base_plus_slg', 'p_era', 'p_quality_start', 'p_ball',
       'p_called_strike', 'p_gnd_into_dp', 'p_swinging_strike',
       'p_total_strike', 'p_total_swinging_strike', 'exit_velocity_av
g',
       'launch_angle_avg', 'sweet_spot_percent', 'barrel',
       'barrel_batted_rate', 'solidcontact_percent', 'hard_hit_percen
t',
       'z_swing_percent', 'z_swing_miss_percent', 'oz_swing_percent',
       'oz_swing_miss_percent', 'oz_contact_percent', 'out_zone_swing_m
iss',
       'out_zone_swing', 'out_zone_percent', 'out_zone',
       'pitch_count_offspeed', 'pitch_count_fastball', 'pitch_count_bre
aking',
       'pitch_count', 'iz_contact_percent', 'in_zone_swing_miss',
       'in_zone_swing', 'in_zone_percent', 'in_zone', 'edge_percent', '
edge',
       'whiff_percent', 'swing_percent'],
      dtype='object')
Index(['Name', 'player_id', 'year', 'p_game', 'Innings', 'pa', 'ab', 'h
it',
       'strikeout', 'walk', 'k_percent', 'bb_percent', 'batting_avg',
       'slugging', 'on_base_percent', 'on_base_plus_slg', 'ERA',
       'p_quality_start', 'p_ball', 'p_called_strike', 'GIDP Forced',
       'p_swinging_strike', 'p_total_strike', 'p_total_swinging_strik
e',
       'exit_velocity_avg', 'launch_angle_avg', 'sweet_spot_percent', '
barrel',
       'barrel_batted_rate', 'solidcontact_percent', 'hard_hit_percen
t',
       'z_swing_percent', 'z_swing_miss_percent', 'oz_swing_percent',
       'oz_swing_miss_percent', 'oz_contact_percent', 'out_zone_swing_m
iss',
       'out_zone_swing', 'out_zone_percent', 'out_zone',
       'pitch_count_offspeed', 'pitch_count_fastball', 'pitch_count_bre
aking',
       'pitch_count', 'iz_contact_percent', 'in_zone_swing_miss',
       'in_zone_swing', 'in_zone_percent', 'in_zone', 'edge_percent', '
edge',
       'whiff_percent', 'swing_percent'],
      dtype='object')
```

Out[432…

	Name	player_id	year	p_game	Innings	pa	ab	hit	strikeout	w
0	Blanco, Ronel	669854	2025	9	48.1	199	176	37	48	
1	Kelly, Merrill	518876	2025	32	184.0	748	696	157	167	
2	Hamel, Dom	687589	2025	1	1.0	6	5	3	0	
3	Basso, Brady	669620	2025	11	11.2	56	47	12	8	
4	Montas, Frankie	593423	2025	9	38.2	178	161	48	32	
...	...	...	...	...	...	...	...	...	...	
804	Henriquez, Edgardo	683618	2025	22	19.0	78	72	17	18	
805	Peraza, Oswald	672724	2025	2	2.1	16	15	9	0	
806	Leclerc, José	600917	2025	10	9.0	46	41	13	8	
807	White, Owen	669391	2025	3	7.0	35	32	14	6	
808	Selby, Colin	681882	2025	11	14.0	60	58	16	14	

809 rows × 53 columns

In [433…

```
#Question 7 - dropped unused columns here to reduce work in later ques
pitch = pitcher.drop(columns=['slugging', 'on_base_percent', 'GIDP For
pitch
print(pitch.columns)
```

```
Index(['Name', 'Innings', 'pa', 'hit', 'strikeout', 'walk', 'batting_av
g',
       'ERA', 'exit_velocity_avg', 'pitch_count_offspeed',
       'pitch_count_fastball', 'pitch_count_breaking', 'pitch_count'],
      dtype='object')
```

In [434…

```
#Question 5 - all parts
print(pitch.columns)
pitch['Name'].describe()
pitch['Name'].unique()
print(pitch['Name'].mode())
#a) This lists the name of each player
#b) Character String
#c) No names duplicate, only appear once - this took up a lot of space
```

```
#print it for final submission. They are printed 'last name, first nam
#d) No missing values

print(pitch['Innings'].describe())
#a) This lists the amount of innings each player pitched
#b) float real numbers
#c) the minimum innings pitched is 1, while the max is 207.0. the mean
#Unique values are not expected here, multiple people could have pitch
#d) No missing values

print(pitch['pa'].describe())
#a)This lists the number of plate appearances (batters faced).
#b) float real numbers
#c) min is 4, max is 856, mean is 225. Unique values are not expected
#d) No missing values

print(pitch['hit'].describe())
#a) This lists the number of hits give up by the player
#b) float real numbers
#c) min is 0, max is 210, mean is 49.31. Unique values are not expecte
#d) No missing values. A value of zero is possible if the pitcher did
#any hits

print(pitch['strikeout'].describe())
#a) This lists the number of strikeouts recorded by each player
#b) float real numbers
#c) min is 0, max is 255, mean is 50.21. Unique values are not expecte
#d) No missing values. A value of zero is possible if the pitcher did
#anyone

print(pitch['walk'].describe())
#a) This lists the number of walks allowed by each player
#b) float real numbers
#c) min is 0, max is 83, mean is 18.94. Unique values are not expected
#d) No missing values. A value of zero is possible if the pitcher did
#anyone

print(pitch['batting_avg'].describe())
#a) This lists the batting average allowed for each pitcher
#b) float real numbers
#c) min is 0.000, max is 0.667, mean is .258. Unique values are not ex
#d) No missing values. A value of zero is possible if the pitcher did

print(pitch['ERA'].describe())
#a) This lists the earned run average per inning for each pitcher
#b) float real numbers
#c) min is 0.000, max is 30.86, mean is 5.04. Unique values are not ex
#d) No missing values. A value of zero is possible if the pitcher did

print(pitch['exit_velocity_avg'].describe())
#a) This lists the average velocity off balls batter off of the pitche
#b) float real numbers
```

```
#c) min is 72.8, max is 98.7, mean is 89.43. Unique values are not exp
#d) No missing values

print(pitch['pitch_count_offspeed'].describe())
#a) This lists the number of offspeed pitches thrown by the pitcher
#b) float real numbers
#c) min is 0, max is 1138, mean is 120.14. Unique values are not expec
#d) No missing values. A value of 0 could occur if the pitcher does no

print(pitch['pitch_count_fastball'].describe())
#a) This lists the number of fastball pitches thrown by the pitcher
#b) float real numbers
#c) min is 0, max is 2514, mean is 482.2. Unique values are not expect
#d) No missing values. A value of 0 could occur if the pitcher does no

print(pitch['pitch_count_breaking'].describe())
#a) This lists the number of breaking pitches thrown by the pitcher
#b) float real numbers
#c) min is 0, max is 1597, mean is 205.81. Unique values are not expec
#d) No missing values. A value of 0 could occur if the pitcher does no

print(pitch['pitch_count'].describe())
#a) This lists the number of total pitches thrown by the pitcher
#b) float real numbers
#c) min is 12, max is 3282, mean is 875.44. Unique values are not expe
#d) No missing values
```

```
Index(['Name', 'Innings', 'pa', 'hit', 'strikeout', 'walk', 'batting_av
g',
       'ERA', 'exit_velocity_avg', 'pitch_count_offspeed',
       'pitch_count_fastball', 'pitch_count_breaking', 'pitch_count'],
      dtype='object')
0         Abbott, Andrew
1             Abel, Mick
2          Abner, Philip
3           Abreu, Bryan
4         Acton, Garrett
             ...
804    Zimmermann, Bruce
805         Zuber, Tyler
806      Zulueta, Yosver
807       de Geus, Brett
808        deGrom, Jacob
Name: Name, Length: 809, dtype: object
count    809.000000
mean      52.859951
std       49.693181
min        1.000000
25%       13.100000
50%       39.200000
75%       69.200000
max      207.000000
Name: Innings, dtype: float64
```

```
count    809.000000
mean     225.297899
std      206.465599
min        4.000000
25%       62.000000
50%      172.000000
75%      291.000000
max      856.000000
Name: pa, dtype: float64
count    809.000000
mean      49.317676
std       46.164918
min        0.000000
25%       14.000000
50%       36.000000
75%       62.000000
max      210.000000
Name: hit, dtype: float64
count    809.000000
mean      50.217553
std       48.181198
min        0.000000
25%       12.000000
50%       36.000000
75%       73.000000
max      255.000000
Name: strikeout, dtype: float64
count    809.000000
mean      18.943140
std       16.169254
min        0.000000
25%        6.000000
50%       16.000000
75%       26.000000
max       83.000000
Name: walk, dtype: float64
count    809.000000
mean       0.257507
std        0.070640
min        0.000000
25%        0.219000
50%        0.250000
75%        0.283000
max        0.667000
Name: batting_avg, dtype: float64
count    809.000000
mean       5.044586
std        3.438534
min        0.000000
25%        3.140000
50%        4.250000
75%        5.910000
```

```
max       30.860000
Name: ERA, dtype: float64
count    809.000000
mean      89.429419
std        2.425432
min       72.800000
25%       88.300000
50%       89.500000
75%       90.600000
max       98.700000
Name: exit_velocity_avg, dtype: float64
count     809.000000
mean      120.142151
std       168.374063
min         0.000000
25%         5.000000
50%        49.000000
75%       169.000000
max      1138.000000
Name: pitch_count_offspeed, dtype: float64
count     809.000000
mean      482.208900
std       458.695243
min         0.000000
25%       128.000000
50%       346.000000
75%       657.000000
max      2514.000000
Name: pitch_count_fastball, dtype: float64
count     809.000000
mean      205.814586
std       252.074046
min         0.000000
25%        18.000000
50%       104.000000
75%       315.000000
max      1597.000000
Name: pitch_count_breaking, dtype: float64
count     809.000000
mean      875.449938
std       799.776384
min        13.000000
25%       239.000000
50%       656.000000
75%      1130.000000
max      3282.000000
Name: pitch_count, dtype: float64
```

In [435…

```
#Question 6

#I will be looking to determine which pitch mix provided the most succ
#on average in 2025. I will do this by comparing the pitch totals thro
#type of pitch to their ERAs, batting averages allowed, average exit v
```

```
#and hits.
#For this, I will look at the 'batting_avg', 'ERA', 'hit', 'exit_veloc
#'pitch_count_offspeed', 'pitch_count_fastball', 'pitch_count_breaking
#'pitch_count' columns

# I will also look at which pitch mixes create the highest strikeout a
#For this, I will look at 'Innings', 'pa', 'strikeout', and 'walk' col

#I can also look to see if there is correlation between high strikeout
#walks per inning to the success factors listed for question 1. All re
#would be used for this. Hence, I am keeping the data in one DataFrame
```

In [436…

```
#Question 8

#For the first and third questions, I could use ANOVA to compare diffe
#For question 1, that would include comparing those who primarily use
#breaking, fastballs vs offspeed, and breaking vs. offspeed. I could c
#allowed. #For question 3, I could compare differences in the same cat
#groups above a certain strikeout number and below a certain walk numb

#For the second question, I could use regression to predict how many w
#strikeouts a pitcher would have with different pitch mixes based on t
#innings that they pitch.
```

In [437…

```
#Project Deliverable 2 Starts Below:
```

In [438…

```
#Part 1 - Handling Missing Data:
#As described in the lab, there is no missing data to handle. All unne
#have already been dropped. No further work is needed for any of the s
#All values that equal zero mean that the event did not occur, which i
#valuable data. No NAs appear.

pitch.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 809 entries, 0 to 808
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Name                  809 non-null    object
 1   Innings               809 non-null    float64
 2   pa                    809 non-null    int64
 3   hit                   809 non-null    int64
 4   strikeout             809 non-null    int64
 5   walk                  809 non-null    int64
 6   batting_avg           809 non-null    float64
 7   ERA                   809 non-null    float64
 8   exit_velocity_avg     809 non-null    float64
 9   pitch_count_offspeed  809 non-null    int64
 10  pitch_count_fastball  809 non-null    int64
 11  pitch_count_breaking  809 non-null    int64
 12  pitch_count           809 non-null    int64
dtypes: float64(4), int64(8), object(1)
memory usage: 82.3+ KB
```

In [439…

```
#Data Transformations:
#Change Column Types:
#As described in the lab, the name column includes string values (whic
#The remaining columns all include float values. Some of the values (p
#walk, pitch_count_offspeed, pitch_count_fastball, pitch_count_breakin
#only come in whole number values, and are therefore showing as intege

#All of these data types are correct and no changes need to be made.
```

In [440…

```
#Remove Duplicates:
any(pitch['Name'].duplicated())

#There are no duplicate names, so no further work needs to be done for
#Duplicate columns were removed when dropping columns not used in the
```

Out[440…

```
False
```

In [441…

```
#Change Units or Add Columns Based on a Mapping
pitch['Strikeout %'] = (pitch['strikeout']/pitch['pa'])*100
pitch['Walk %'] = (pitch['walk']/pitch['pa'])*100
pitch['Hits per PA'] = (pitch['hit']/pitch['pa'])
pitch['Fastball Use %'] = (pitch['pitch_count_fastball']/pitch['pitch_
pitch['Breaking Use %'] = (pitch['pitch_count_breaking']/pitch['pitch_
pitch['Offspeed Use %'] = (pitch['pitch_count_offspeed']/pitch['pitch_
pitch = pitch.drop(columns = ['hit','strikeout', 'walk','pitch_count_f
pitch
```

Out[441…

	Name	Innings	pa	batting_avg	ERA	exit_velocity_avg	pitch_cou
0	Blanco, Ronel	48.1	199	0.210	4.10	85.9	79
1	Kelly, Merrill	184.0	748	0.226	3.52	90.2	283
2	Hamel, Dom	1.0	6	0.600	0.00	91.9	1
3	Basso, Brady	11.2	56	0.255	2.31	89.8	20
4	Montas, Frankie	38.2	178	0.298	6.28	88.5	70
...	...	...	...	...	...	...	
804	Henriquez, Edgardo	19.0	78	0.236	2.37	88.2	27
805	Peraza, Oswald	2.1	16	0.600	30.86	90.5	4
806	Leclerc, José	9.0	46	0.317	6.00	88.8	22
807	White, Owen	7.0	35	0.438	9.00	91.1	11
808	Selby, Colin	14.0	60	0.276	3.21	90.8	21

809 rows × 13 columns

In [442…

```
#Replace Values - No need to complete this step, as all columns alread
#values.
```

In [443…

```
#Relabel Rows and Columns
#I am deciding to keep the names in to more easily identify certain ro
#Because of this, I am going to reset the row index to the names.
pitch = pitch.set_index('Name')

#Columns were renamed in the lab already. Further adjustments were mad
#of this deliverable.

pitch
```

Out[443…

Name	Innings	pa	batting_avg	ERA	exit_velocity_avg	pitch_count	S
Blanco, Ronel	48.1	199	0.210	4.10	85.9	794	2
Kelly, Merrill	184.0	748	0.226	3.52	90.2	2833	2
Hamel, Dom	1.0	6	0.600	0.00	91.9	13	(
Basso, Brady	11.2	56	0.255	2.31	89.8	206	1
Montas, Frankie	38.2	178	0.298	6.28	88.5	708	1
...	...	...	...	...	...	...	
Henriquez, Edgardo	19.0	78	0.236	2.37	88.2	279	2
Peraza, Oswald	2.1	16	0.600	30.86	90.5	41	(
Leclerc, José	9.0	46	0.317	6.00	88.8	229	1
White, Owen	7.0	35	0.438	9.00	91.1	119	1
Selby, Colin	14.0	60	0.276	3.21	90.8	214	2

809 rows × 12 columns

In [444…

```
#Bin Data:
#Asked the following prompt to ChatGPT - 'How to include 0 in a bin us
#Since I am did not actually utilize any data from the dataset, there
#protection needed while using AI in this case.

#After doing the bins, the use % columns become less relevant. I will
#for now in case I want to look into different cutoffs in the future.

#Also some pitchers do not utilize a certain type of pitch. This is no
#be excluded from the data and will be instead label as 'No Use'. Thei
#can still be measured in testing and the lack of use is valuable to d
#importance of the pitch type to success.

import numpy as np
bins = [-1,0,25,50,np.inf]
names = ['No Use','Minimal Use','Moderate Use','Primary Pitch']
```

```
pitch['Use of Fastball'] = pd.cut(pitch['Fastball Use %'], bins, label
pitch['Use of Breaking'] = pd.cut(pitch['Breaking Use %'], bins, label
pitch['Use of Offspeed'] = pd.cut(pitch['Offspeed Use %'], bins, label
pitch
```

Out[444…

Name	Innings	pa	batting_avg	ERA	exit_velocity_avg	pitch_count	S
Blanco, Ronel	48.1	199	0.210	4.10	85.9	794	2
Kelly, Merrill	184.0	748	0.226	3.52	90.2	2833	2
Hamel, Dom	1.0	6	0.600	0.00	91.9	13	(
Basso, Brady	11.2	56	0.255	2.31	89.8	206	1
Montas, Frankie	38.2	178	0.298	6.28	88.5	708	1
...	...	...	...	...	...	...	
Henriquez, Edgardo	19.0	78	0.236	2.37	88.2	279	2
Peraza, Oswald	2.1	16	0.600	30.86	90.5	41	(
Leclerc, José	9.0	46	0.317	6.00	88.8	229	1
White, Owen	7.0	35	0.438	9.00	91.1	119	1
Selby, Colin	14.0	60	0.276	3.21	90.8	214	2

809 rows × 15 columns

In [445…

```
#Detect and Filter Outliers:
#For my data, pitchers wo pitched very little may skew the data. There
#going to remove pitchers who did not face at least 75 plate appearanc
pitch = pitch[pitch['pa'] >= 75]
pitch
```

Out[445…

Name	Innings	pa	batting_avg	ERA	exit_velocity_avg	pitch_count	S
Blanco, Ronel	48.1	199	0.210	4.10	85.9	794	24
Kelly, Merrill	184.0	748	0.226	3.52	90.2	2833	22.
Montas, Frankie	38.2	178	0.298	6.28	88.5	708	17
Schultz, Paxton	24.2	110	0.276	4.38	91.1	448	25.
Dreyer, Jack	76.1	307	0.203	2.95	90.3	1146	24
...	...	...	...	...	...	...	
Junk, Janson	110.0	447	0.266	4.17	91.6	1622	17
Martin, Davis	142.2	601	0.251	4.10	90.7	2309	17.
Junis, Jakob	66.2	274	0.257	2.97	87.6	1032	20
Márquez, Germán	126.1	591	0.317	6.70	91.7	2085	14.
Henriquez, Edgardo	19.0	78	0.236	2.37	88.2	279	23

576 rows × 15 columns

In [446…

```
#Create a Sample - I think the analysis should be able to handle the 5
#Therefore, this step is not applicable.
```

In [447…

```
#Compute Dummy Columns for Categorical Variables - Because all of my d
#names is numerical and names is only being used for identification, t
#need for this step.
```

In [448…

```
#Process String Columns - Because all data is numerical except for the
#identification, there is no need for this step. If one wanted to, we
#changed the names from 'last name, first name' order to 'First Last'.
#is not providing any benefit to me for the data.
```

In [449…

```
#Relabel Columns
#At this point, I do want to reorder the columns, remove underscores a
pitch = pd.DataFrame(pitch, columns=['Innings','pa','pitch_count','Hit
```

```
pitch = pitch.rename(columns={'pa':'PA','pitch_count':'Pitch Count','b
pitch
```

Out[449…

Name	Innings	PA	Pitch Count	Hits per PA	Batting AVG	ERA	Exit Velo AVG	Strikeout %	V
Blanco, Ronel	48.1	199	794	0.185930	0.210	4.10	85.9	24.120603	10.
Kelly, Merrill	184.0	748	2833	0.209893	0.226	3.52	90.2	22.326203	6
Montas, Frankie	38.2	178	708	0.269663	0.298	6.28	88.5	17.977528	7.8
Schultz, Paxton	24.2	110	448	0.245455	0.276	4.38	91.1	25.454545	7.
Dreyer, Jack	76.1	307	1146	0.182410	0.203	2.95	90.3	24.104235	7.
...	...	...	...	...	...	...	...	...	
Junk, Janson	110.0	447	1622	0.250559	0.266	4.17	91.6	17.225951	2.9
Martin, Davis	142.2	601	2309	0.226290	0.251	4.10	90.7	17.304493	7.9
Junis, Jakob	66.2	274	1032	0.233577	0.257	2.97	87.6	20.072993	6.5
Márquez, Germán	126.1	591	2085	0.284264	0.317	6.70	91.7	14.043993	8.
Henriquez, Edgardo	19.0	78	279	0.217949	0.236	2.37	88.2	23.076923	6.

576 rows × 15 columns

In [450…

```
#2/22/2026 - Project Assignment 3 - starting by resetting index:
pitch = pitch.reset_index(level=0)
pitch
```

Out[450…

	Name	Innings	PA	Pitch Count	Hits per PA	Batting AVG	ERA	Exit Velo AVG	Strikeout %
0	Blanco, Ronel	48.1	199	794	0.185930	0.210	4.10	85.9	24.120603
1	Kelly, Merrill	184.0	748	2833	0.209893	0.226	3.52	90.2	22.326203
2	Montas, Frankie	38.2	178	708	0.269663	0.298	6.28	88.5	17.977528
3	Schultz, Paxton	24.2	110	448	0.245455	0.276	4.38	91.1	25.454545
4	Dreyer, Jack	76.1	307	1146	0.182410	0.203	2.95	90.3	24.104235
...	...	...	...	...	...	...	...	...	...
571	Junk, Janson	110.0	447	1622	0.250559	0.266	4.17	91.6	17.225951
572	Martin, Davis	142.2	601	2309	0.226290	0.251	4.10	90.7	17.304493
573	Junis, Jakob	66.2	274	1032	0.233577	0.257	2.97	87.6	20.072993
574	Márquez, Germán	126.1	591	2085	0.284264	0.317	6.70	91.7	14.043993
575	Henriquez, Edgardo	19.0	78	279	0.217949	0.236	2.37	88.2	23.076923

576 rows × 16 columns

In [451…

```
#Processing Strings -
#All strings are within dataset, so no individual manipulation with bui
#functions needed.

#Manipulating strings with pandas:
#Adjusting names to be 'First Last' instead of 'Last, First'
pitch[ ['Last', 'First'] ] = pitch['Name'].str.split(',', expand=True)
pitch['Full Name'] = pitch['First'] + ' ' + pitch['Last']
pitch = pitch.drop(columns=['Name', 'First', 'Last'])
pitch = pitch.set_index(['Full Name'])
pitch
```

Out[451…

Full Name	Innings	PA	Pitch Count	Hits per PA	Batting AVG	ERA	Exit Velo AVG	Strikeout %	W
Ronel Blanco	48.1	199	794	0.185930	0.210	4.10	85.9	24.120603	10.0
Merrill Kelly	184.0	748	2833	0.209893	0.226	3.52	90.2	22.326203	6.
Frankie Montas	38.2	178	708	0.269663	0.298	6.28	88.5	17.977528	7.8
Paxton Schultz	24.2	110	448	0.245455	0.276	4.38	91.1	25.454545	7.2
Jack Dreyer	76.1	307	1146	0.182410	0.203	2.95	90.3	24.104235	7.8
...	...	...	...	...	...	...	...	...	
Janson Junk	110.0	447	1622	0.250559	0.266	4.17	91.6	17.225951	2.9
Davis Martin	142.2	601	2309	0.226290	0.251	4.10	90.7	17.304493	7.9
Jakob Junis	66.2	274	1032	0.233577	0.257	2.97	87.6	20.072993	6.5
Germán Márquez	126.1	591	2085	0.284264	0.317	6.70	91.7	14.043993	8.1
Edgardo Henriquez	19.0	78	279	0.217949	0.236	2.37	88.2	23.076923	6.4

576 rows × 15 columns

In [452…

```
#All other data is float, integer, or categorical and does not need to
#in any way. Therefore, no further work is required for processing str
```

In [453…

```
#Combining and merging datasets -
#'pitch type vs performance.csv' is the only data being used for this
#that combining and merging data is not applicable to this project.
```

In [454…

```
#Reshaping and Pivoting - Below, I pivoted my dataframe to place the E
#into separate columns based on the usage of each type of pitch.

#From here, I cleaned the data to remove pitchers who did not use the
#at least a moderate amount. In this process, I utilized ChatGPT, aski
#"How would I write code for python that would remove any row in a dat
#NA for both of two columns?". Again, I did not upload my data, so the
#privacy concerns using an open AI source.
```

```
#I calculated the mean for moderate and primary usage for each pitch t
#to analysis.

#There is no need to reshape anything other than what was done in the
#review, please let me know if you have any suggestions that will be h
#regarding the application of reshaping/pivoting.

pitch = pitch.reset_index(level=0)
```

In [455…

```
#Shows ERA for each usage amount of fastball pitches - the mean is the
#pitchers who primarily throw fastballs.
fastball_era = pitch.pivot(index='Full Name', columns='Use of Fastball
fastball_era = fastball_era.drop(columns=['Minimal Use'])
fastball_era = fastball_era.dropna(subset=['Moderate Use', 'Primary Pi
print(fastball_era)
print(fastball_era['Primary Pitch'].mean())
print(fastball_era['Moderate Use'].mean())
```

```
Use of Fastball   Moderate Use  Primary Pitch
Full Name
AJ Blubaugh                NaN           1.69
AJ Smith-Shawver          3.86            NaN
Aaron Ashby                NaN           2.16
Aaron Bummer              3.81            NaN
Aaron Civale               NaN           4.85
...                        ...            ...
Zack Kelly                 NaN           4.58
Zack Littell              3.81            NaN
Zack Wheeler               NaN           2.71
Zak Kent                  4.58            NaN
Zebby Matthews             NaN           5.56

[573 rows x 2 columns]
4.34312661498708
4.179516129032258
```

In [456…

```
#Shows ERA for each usage amount of breaking pitches - the mean is the
#pitchers who primarily throw breaking pitches.
breaking_era = pitch.pivot(index='Full Name', columns='Use of Breaking
breaking_era = breaking_era.drop(columns=['Minimal Use', 'No Use'])
breaking_era = breaking_era.dropna(subset=['Moderate Use', 'Primary Pi
print(breaking_era)
print(breaking_era['Primary Pitch'].mean())
print(breaking_era['Moderate Use'].mean())
```

```
Use of Breaking  Moderate Use  Primary Pitch
Full Name
Aaron Ashby              2.16            NaN
Aaron Bummer             3.81            NaN
Aaron Civale             4.85            NaN
Aaron Nola               6.01            NaN
Abner Uribe              1.67            NaN
...                       ...            ...
Yusei Kikuchi             NaN           3.99
Zac Gallen               4.83            NaN
Zack Littell             3.81            NaN
Zak Kent                  NaN           4.58
Zebby Matthews           5.56            NaN

[268 rows x 2 columns]
3.86
4.328454935622317
```

In [457…

```
#Shows ERA for each usage amount of offspeed pitches - the mean is the
#pitchers who primarily throw offspeed pitches.
offspeed_era = pitch.pivot(index='Full Name', columns='Use of Offspeed
offspeed_era = offspeed_era.drop(columns=['Minimal Use', 'No Use'])
offspeed_era = offspeed_era.dropna(subset=['Moderate Use', 'Primary Pi
print(offspeed_era)
print(offspeed_era['Primary Pitch'].mean())
print(offspeed_era['Moderate Use'].mean())
```

```
Use of Offspeed      Moderate Use  Primary Pitch
Full Name
AJ Smith-Shawver             3.86            NaN
Alek Jacob                   5.13            NaN
Angel Chivilli               7.06            NaN
Bailey Ober                  5.10            NaN
Brandon Eisert               4.39            NaN
...                           ...            ...
Yaramil Hiraldo              4.58            NaN
Yerry De los Santos          3.28            NaN
Yoshinobu Yamamoto           2.49            NaN
Yuki Matsui                  3.98            NaN
Zack Littell                 3.81            NaN

[81 rows x 2 columns]
5.07
4.096493506493506
```

In [458…

```
#Shows Walk % for each usage amount of fastball pitches - the mean is
#batters walked for pitchers who primarily throw fastballs.
fastball_walk = pitch.pivot(index='Full Name', columns='Use of Fastbal
fastball_walk = fastball_walk.drop(columns=['Minimal Use'])
fastball_walk = fastball_walk.dropna(subset=['Moderate Use', 'Primary
print(fastball_walk)
print(fastball_walk['Primary Pitch'].mean())
print(fastball_walk['Moderate Use'].mean())
```

```
Use of Fastball   Moderate Use  Primary Pitch
Full Name
AJ Blubaugh                NaN       9.600000
AJ Smith-Shawver     10.471204            NaN
Aaron Ashby                NaN       8.888889
Aaron Bummer          7.627119            NaN
Aaron Civale               NaN       7.586207
...                        ...            ...
Zack Kelly                 NaN       7.894737
Zack Littell          4.216074            NaN
Zack Wheeler               NaN       5.641026
Zak Kent             10.526316            NaN
Zebby Matthews             NaN       6.779661

[573 rows x 2 columns]
8.741194694930963
8.919412214975717
```

In [459…

```
#Shows Walk % for each usage amount of breaking pitches - the mean is
#batters walked for pitchers who primarily throw breaking pitches.
breaking_walk = pitch.pivot(index='Full Name', columns='Use of Breakin
breaking_walk = breaking_walk.drop(columns=['Minimal Use', 'No Use'])
breaking_walk = breaking_walk.dropna(subset=['Moderate Use', 'Primary
print(breaking_walk)
print(breaking_walk['Primary Pitch'].mean())
print(breaking_walk['Moderate Use'].mean())
```

```
Use of Breaking  Moderate Use  Primary Pitch
Full Name
Aaron Ashby          8.888889            NaN
Aaron Bummer         7.627119            NaN
Aaron Civale         7.586207            NaN
Aaron Nola           6.930693            NaN
Abner Uribe          9.060403            NaN
...                       ...            ...
Yusei Kikuchi            NaN       9.585492
Zac Gallen           8.118081            NaN
Zack Littell         4.216074            NaN
Zak Kent                 NaN      10.526316
Zebby Matthews       6.779661            NaN

[268 rows x 2 columns]
9.236655165937503
8.957025992991133
```

In [460…

```
#Shows Walk % for each usage amount of offspeed pitches - the mean is
#batters walked for pitchers who primarily throw offspeed pitches.
offspeed_walk = pitch.pivot(index='Full Name', columns='Use of Offspee
offspeed_walk = offspeed_walk.drop(columns=['Minimal Use', 'No Use'])
offspeed_walk = offspeed_walk.dropna(subset=['Moderate Use', 'Primary
print(offspeed_walk)
print(offspeed_walk['Primary Pitch'].mean())
print(offspeed_walk['Moderate Use'].mean())
```

```
Use of Offspeed      Moderate Use  Primary Pitch
Full Name
AJ Smith-Shawver        10.471204            NaN
Alek Jacob               9.150327            NaN
Angel Chivilli           8.363636            NaN
Bailey Ober              4.967949            NaN
Brandon Eisert           8.169935            NaN
...                           ...            ...
Yaramil Hiraldo         10.714286            NaN
Yerry De los Santos     10.625000            NaN
Yoshinobu Yamamoto       8.625731            NaN
Yuki Matsui             12.132353            NaN
Zack Littell             4.216074            NaN

[81 rows x 2 columns]
12.57711277233749
8.909564250398528
```

In [461…

```
#Shows Strikeout % for each usage amount of fastball pitches - the mea
#batters struck out for pitchers who primarily throw fastballs.
fastball_k = pitch.pivot(index='Full Name', columns='Use of Fastball',
fastball_k = fastball_k.drop(columns=['Minimal Use'])
fastball_k = fastball_k.dropna(subset=['Moderate Use', 'Primary Pitch'
print(fastball_k)
print(fastball_k['Primary Pitch'].mean())
print(fastball_k['Moderate Use'].mean())
```

```
Use of Fastball   Moderate Use  Primary Pitch
Full Name
AJ Blubaugh                NaN      28.000000
AJ Smith-Shawver     21.989529            NaN
Aaron Ashby                NaN      28.148148
Aaron Bummer         21.610169            NaN
Aaron Civale               NaN      20.229885
...                        ...            ...
Zack Kelly                 NaN      23.026316
Zack Littell         17.127800            NaN
Zack Wheeler               NaN      33.333333
Zak Kent             21.052632            NaN
Zebby Matthews             NaN      24.858757

[573 rows x 2 columns]
22.29422747600646
22.719233053885986
```

In [462…

```
#Shows Strikeout % for each usage amount of breaking pitches - the mea
#batters struck out for pitchers who primarily throw breaking pitches.
breaking_k = pitch.pivot(index='Full Name', columns='Use of Breaking',
breaking_k = breaking_k.drop(columns=['Minimal Use', 'No Use'])
breaking_k = breaking_k.dropna(subset=['Moderate Use', 'Primary Pitch'
print(breaking_k)
print(breaking_k['Primary Pitch'].mean())
print(breaking_k['Moderate Use'].mean())
```

```
Use of Breaking  Moderate Use  Primary Pitch
Full Name
Aaron Ashby         28.148148           NaN
Aaron Bummer        21.610169           NaN
Aaron Civale        20.229885           NaN
Aaron Nola          24.009901           NaN
Abner Uribe         30.201342           NaN
...                       ...           ...
Yusei Kikuchi             NaN     22.538860
Zac Gallen          21.525215           NaN
Zack Littell        17.127800           NaN
Zak Kent                  NaN     21.052632
Zebby Matthews      24.858757           NaN

[268 rows x 2 columns]
24.08059207111322
22.84456731263239
```

In [463…

```
#Shows Strikeout % for each usage amount of offspeed pitches - the mea
#batters struck out for pitchers who primarily throw offspeed pitches.
offspeed_k = pitch.pivot(index='Full Name', columns='Use of Offspeed',
offspeed_k = offspeed_k.drop(columns=['Minimal Use', 'No Use'])
offspeed_k = offspeed_k.dropna(subset=['Moderate Use', 'Primary Pitch'
print(offspeed_k)
print(offspeed_k['Primary Pitch'].mean())
print(offspeed_k['Moderate Use'].mean())
```

```
Use of Offspeed      Moderate Use  Primary Pitch
Full Name
AJ Smith-Shawver        21.989529            NaN
Alek Jacob              15.032680            NaN
Angel Chivilli          15.636364            NaN
Bailey Ober             19.230769            NaN
Brandon Eisert          24.183007            NaN
...                           ...            ...
Yaramil Hiraldo         25.000000            NaN
Yerry De los Santos     17.500000            NaN
Yoshinobu Yamamoto      29.385965            NaN
Yuki Matsui             22.426471            NaN
Zack Littell            17.127800            NaN

[81 rows x 2 columns]
27.482542990969957
22.50480744698302
```

In [464…

```
#Project Assignment 4 Starts Below

import matplotlib.pyplot as plt

#Because I am not looking into one numerical value independently, I di
#a purpose for using a histogram.

#I did not find it necessary to use a bar chart either. The only time
```

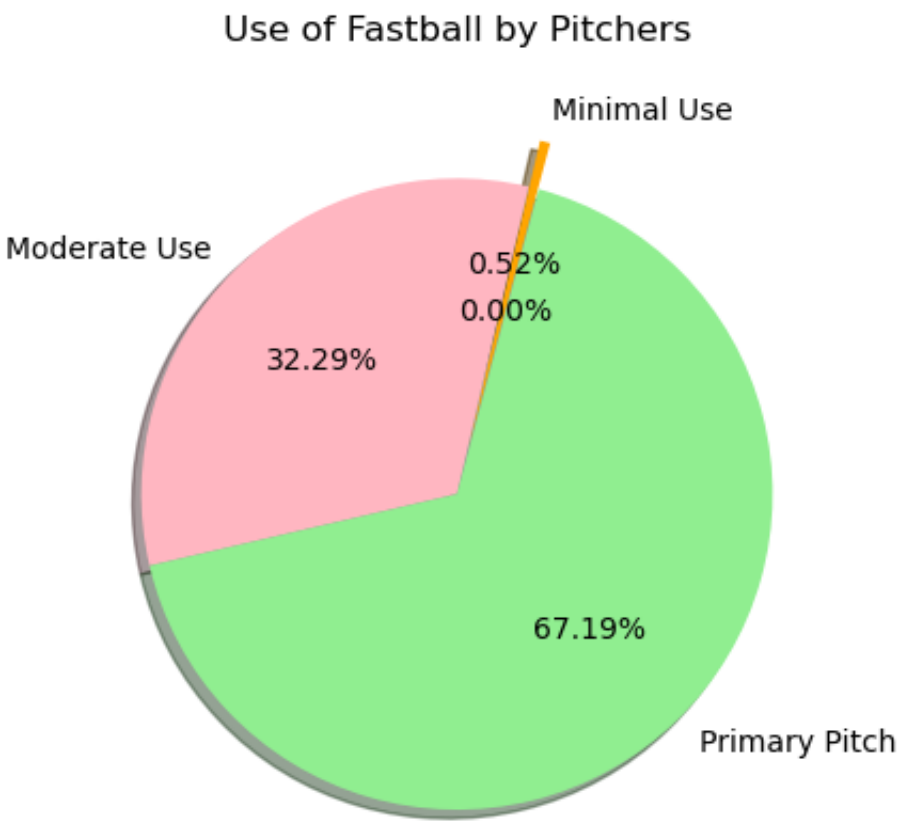
```
#categorical variable was when looking at the % of pitchers who use di
#types. I did this in a pie chart. When comparing one numerical value
#categorical value, I decided to use a boxplot because I wanted to loo
#spread.

#I created the boxplots using seaborn and matplotlib.
#I created the pie charts and scatter plots using pandas and matplotli

#The "analysis" I do explains initial trends I am seeing looking at th
#visualizations. This of course must be looked into further in the nex
```

In [465…

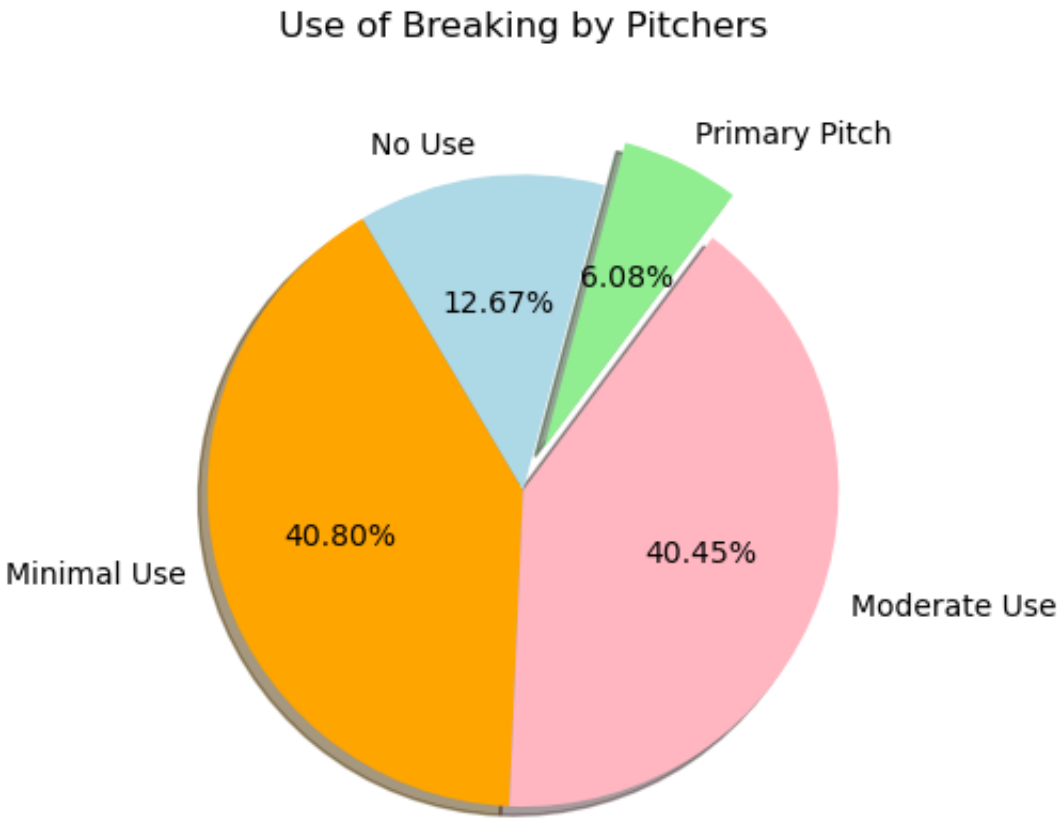
```
#Creating pie charts to gain a better understanding of how many pitche
#pitch type. Since this is looking at one categorical variable and its
#a pie chart makes the most sense.
pie = pitch.groupby('Use of Fastball').size()
pie = pd.DataFrame(pie)
expl=(0,0.15,0,0)
pie.plot(kind='pie', y=0, legend=False, label='',
        autopct='%1.2f%%',
        colors=['lightblue','orange','lightpink','lightgreen'],
        shadow=True,
        startangle=75,
        explode=expl
)
plt.title('Use of Fastball by Pitchers', pad=20)
plt.show()
#Note: All pitchers use fastballs to at least a minimal extent. The 0.
#that.


pie2 = pitch.groupby('Use of Breaking').size()
pie2 = pd.DataFrame(pie2)
expl=(0,0,0,0.15)
pie2.plot(kind='pie', y=0, legend=False, label='',
        autopct='%1.2f%%',
        colors=['lightblue','orange','lightpink','lightgreen'],
        shadow=True,
        startangle=75,
        explode=expl
)
plt.title('Use of Breaking by Pitchers', pad=20)
plt.show()

pie3 = pitch.groupby('Use of Offspeed').size()
pie3 = pd.DataFrame(pie3)
expl=(0,0,0,0.15)
pie3.plot(kind='pie', y=0, legend=False, label='',
        autopct='%1.2f%%',
        colors=['lightblue','orange','lightpink','lightgreen'],
        shadow=True,
        startangle=75,
        explode=expl
```

```
)
plt.title('Use of Offspeed by Pitchers', pad=20)
plt.show()

#Analysis of visualizations:
#Almost all pitchers use fastballs at least a moderate amount.
#Most pitchers use breaking pitches some, but not as a primary pitch.
#Most pitchers only use offspeed pitches a minimally, with a portion n
#using at all.
```

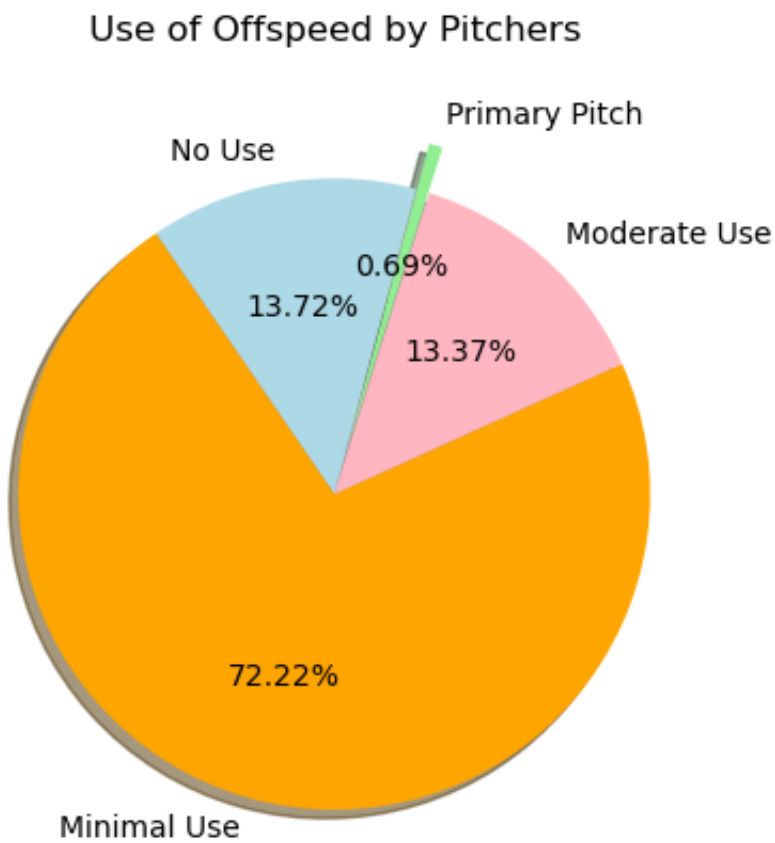
```
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/3637687
436.py:4: FutureWarning: The default of observed=False is deprecated an
d will be changed to True in a future version of pandas. Pass observed=
False to retain current behavior or observed=True to adopt the future d
efault and silence this warning.
  pie = pitch.groupby('Use of Fastball').size()
```




```
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/3637687
436.py:20: FutureWarning: The default of observed=False is deprecated a
nd will be changed to True in a future version of pandas. Pass observed
=False to retain current behavior or observed=True to adopt the future
default and silence this warning.
  pie2 = pitch.groupby('Use of Breaking').size()
```




```
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/3637687
436.py:33: FutureWarning: The default of observed=False is deprecated a
nd will be changed to True in a future version of pandas. Pass observed
=False to retain current behavior or observed=True to adopt the future
default and silence this warning.
  pie3 = pitch.groupby('Use of Offspeed').size()
```

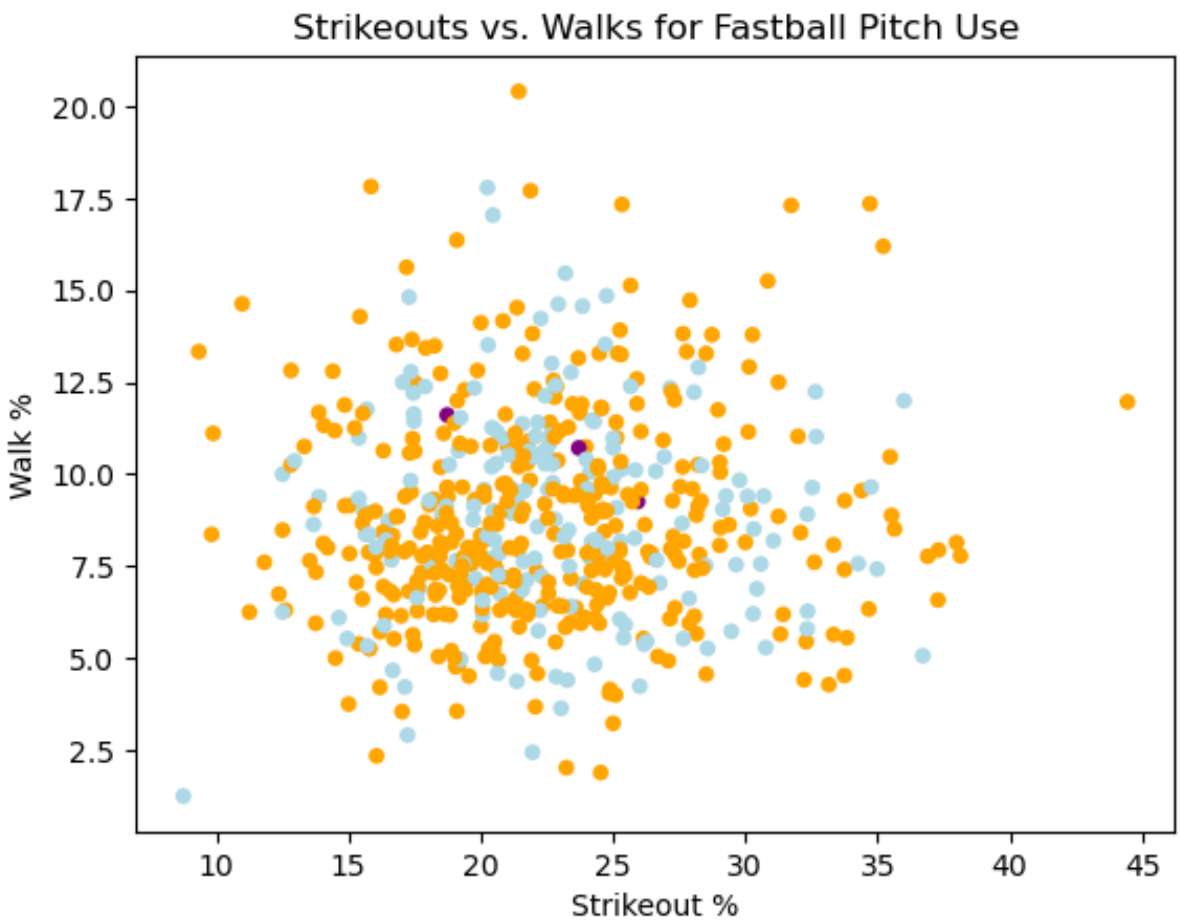


In [466…

```
#Using pandas to create a scatter plot comparing strikeout and walk ra
#fastball use to color code - I am comparing two numerical values and
#value, so scatterplots make the most sense.
colors = pitch['Use of Fastball'].copy()
colors.replace('No Use','lightgreen', inplace=True)
colors.replace('Minimal Use','purple', inplace=True)
colors.replace('Moderate Use','lightblue', inplace=True)
colors.replace('Primary Pitch','orange', inplace=True)

pitch.plot(kind='scatter',
```

```
               x = 'Strikeout %',
               y = 'Walk %',
               c = colors)

plt.title('Strikeouts vs. Walks for Fastball Pitch Use')
plt.xlabel('Strikeout %')
plt.ylabel('Walk %')

plt.show()

#Analysis: there doesn't seem to be much variation between the typical
#but there are more outliers in both strikeouts and walks for primary
#pitchers.
```

```
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/9034344
09.py:5: FutureWarning: The behavior of Series.replace (and DataFrame.r
eplace) with CategoricalDtype is deprecated. In a future version, repla
ce will only be used for cases that preserve the categories. To change
the categories, use ser.cat.rename_categories instead.
  colors.replace('No Use','lightgreen', inplace=True)
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/9034344
09.py:6: FutureWarning: The behavior of Series.replace (and DataFrame.r
eplace) with CategoricalDtype is deprecated. In a future version, repla
ce will only be used for cases that preserve the categories. To change
the categories, use ser.cat.rename_categories instead.
  colors.replace('Minimal Use','purple', inplace=True)
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/9034344
09.py:7: FutureWarning: The behavior of Series.replace (and DataFrame.r
eplace) with CategoricalDtype is deprecated. In a future version, repla
ce will only be used for cases that preserve the categories. To change
the categories, use ser.cat.rename_categories instead.
  colors.replace('Moderate Use','lightblue', inplace=True)
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/9034344
09.py:8: FutureWarning: The behavior of Series.replace (and DataFrame.r
eplace) with CategoricalDtype is deprecated. In a future version, repla
ce will only be used for cases that preserve the categories. To change
the categories, use ser.cat.rename_categories instead.
  colors.replace('Primary Pitch','orange', inplace=True)
```

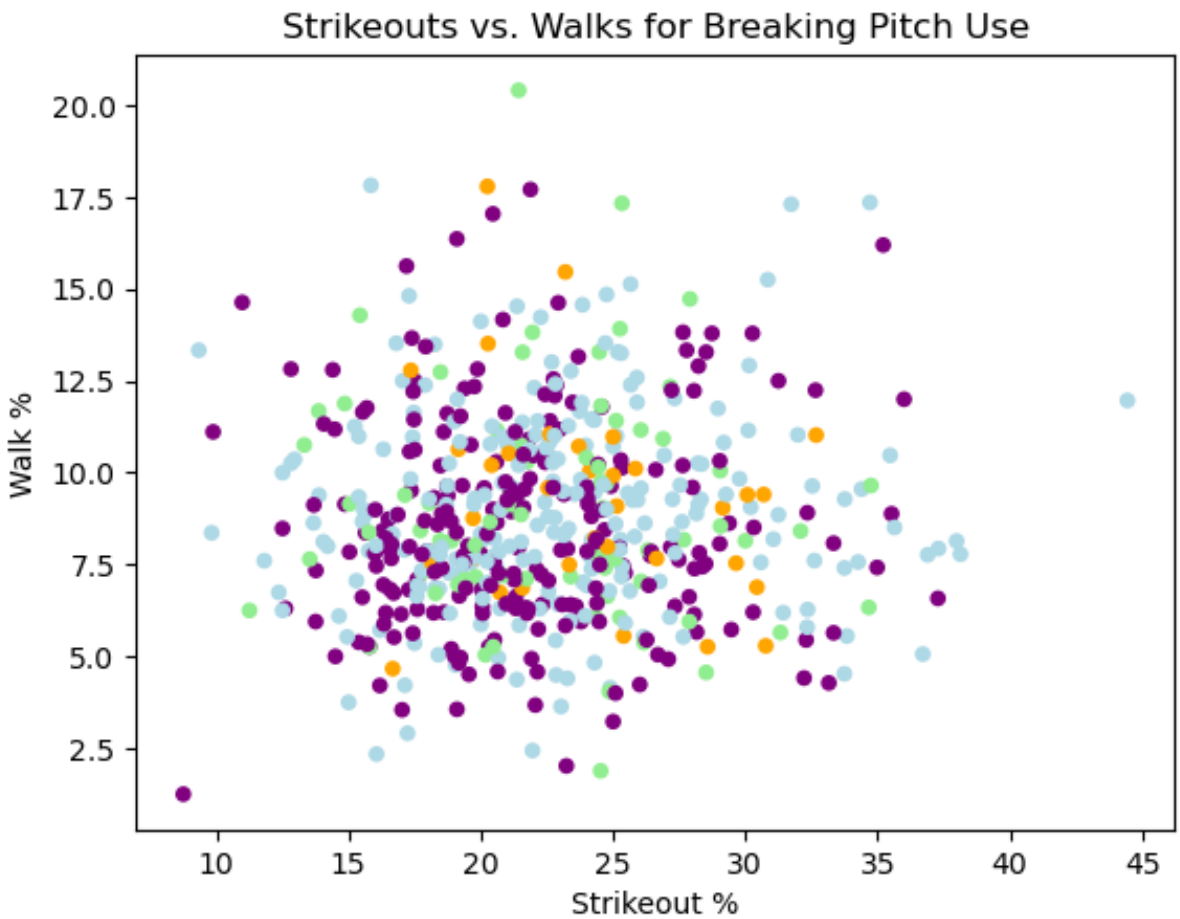


In [467…

```
#Using pandas to create a scatter plot comparing strikeout and walk ra
#breaking use to color code - I am comparing two numerical values and
#value, so scatterplots make the most sense.
colors = pitch['Use of Breaking'].copy()
colors.replace('No Use','lightgreen', inplace=True)
colors.replace('Minimal Use','purple', inplace=True)
colors.replace('Moderate Use','lightblue', inplace=True)
colors.replace('Primary Pitch','orange', inplace=True)

pitch.plot(kind='scatter',
```

```
               x = 'Strikeout %',
               y = 'Walk %',
               c = colors)

plt.title('Strikeouts vs. Walks for Breaking Pitch Use')
plt.xlabel('Strikeout %')
plt.ylabel('Walk %')

plt.show()

#Analysis: Minimal breaking pitch use (purple) seems to strikeout play
#on average, but are also walking batters a bit less than the other gr
```

```
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/3615153
960.py:5: FutureWarning: The behavior of Series.replace (and DataFrame.
replace) with CategoricalDtype is deprecated. In a future version, repl
ace will only be used for cases that preserve the categories. To change
the categories, use ser.cat.rename_categories instead.
  colors.replace('No Use','lightgreen', inplace=True)
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/3615153
960.py:6: FutureWarning: The behavior of Series.replace (and DataFrame.
replace) with CategoricalDtype is deprecated. In a future version, repl
ace will only be used for cases that preserve the categories. To change
the categories, use ser.cat.rename_categories instead.
  colors.replace('Minimal Use','purple', inplace=True)
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/3615153
960.py:7: FutureWarning: The behavior of Series.replace (and DataFrame.
replace) with CategoricalDtype is deprecated. In a future version, repl
ace will only be used for cases that preserve the categories. To change
the categories, use ser.cat.rename_categories instead.
  colors.replace('Moderate Use','lightblue', inplace=True)
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/3615153
960.py:8: FutureWarning: The behavior of Series.replace (and DataFrame.
replace) with CategoricalDtype is deprecated. In a future version, repl
ace will only be used for cases that preserve the categories. To change
the categories, use ser.cat.rename_categories instead.
  colors.replace('Primary Pitch','orange', inplace=True)
```

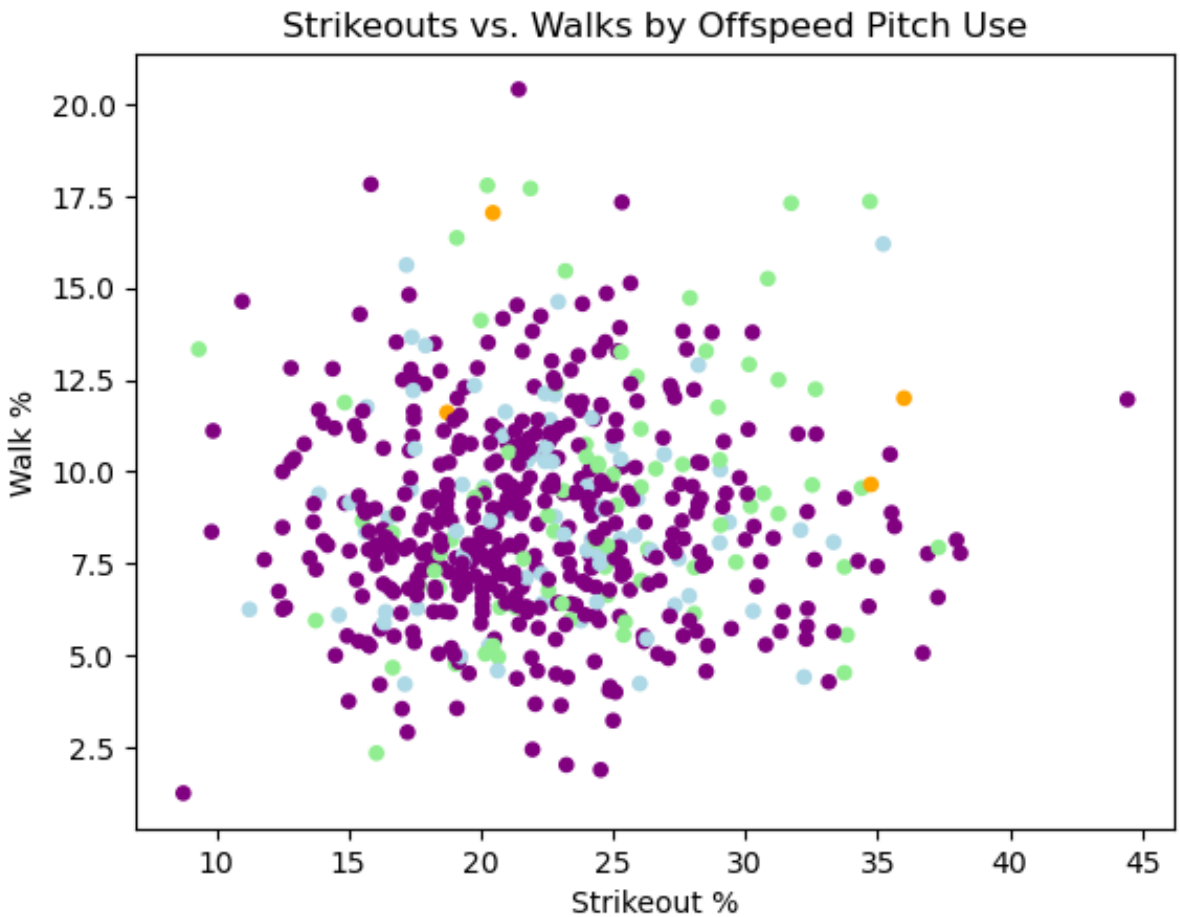


In [468…

```
#Using pandas to create a scatter plot comparing strikeout and walk ra
#offspeed use to color code - I am comparing two numerical values and
#value, so scatterplots make the most sense.
colors = pitch['Use of Offspeed'].copy()
colors.replace('No Use','lightgreen', inplace=True)
colors.replace('Minimal Use','purple', inplace=True)
colors.replace('Moderate Use','lightblue', inplace=True)
colors.replace('Primary Pitch','orange', inplace=True)
```

```
pitch.plot(kind='scatter',
              x = 'Strikeout %',
              y = 'Walk %',
              c = colors)

plt.title('Strikeouts vs. Walks by Offspeed Pitch Use')
plt.xlabel('Strikeout %')
plt.ylabel('Walk %')

plt.show()

#Analysis: Those who are not using offspeed tend to be striking out an
#batters more. This may be due to a breaking pitch with more movement
#high velocity, low control pitching style.
```

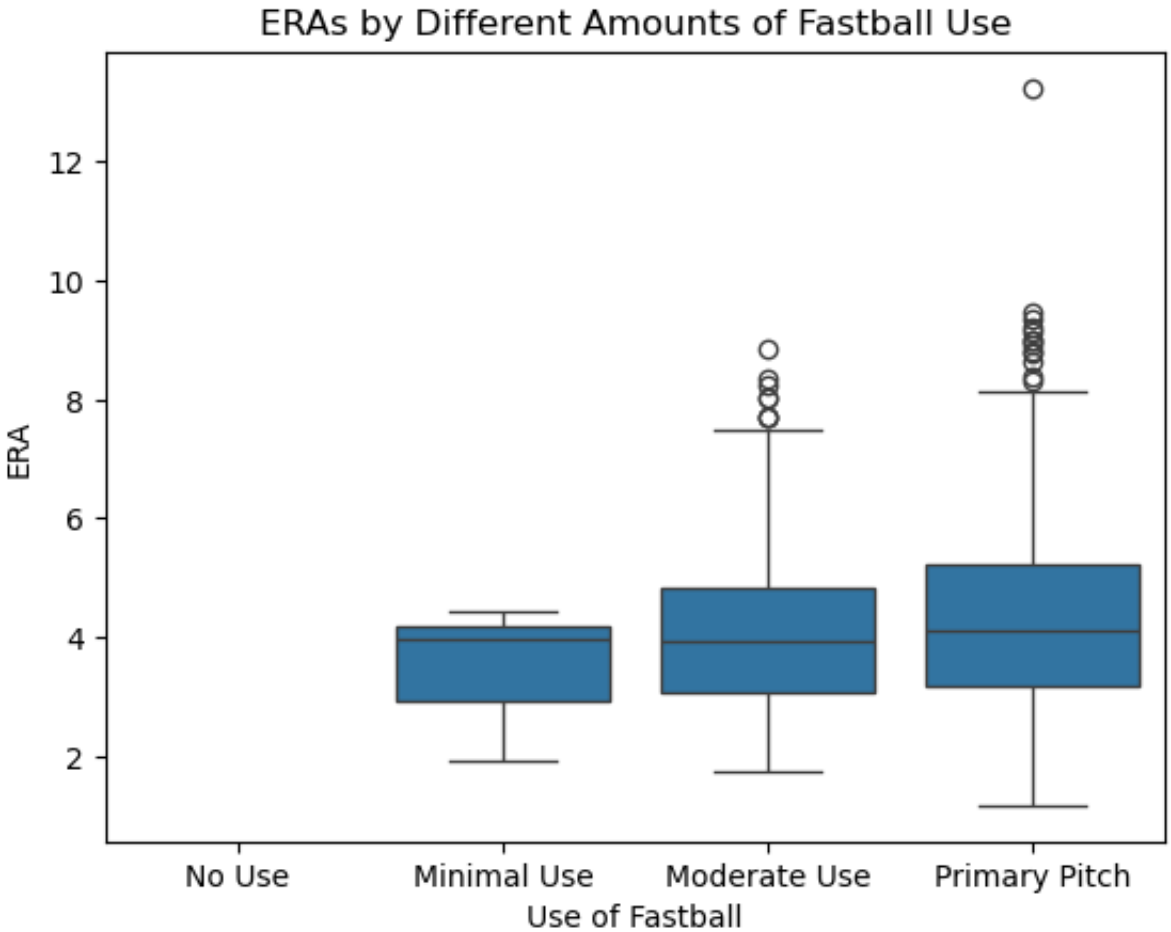
```
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/8983811
37.py:5: FutureWarning: The behavior of Series.replace (and DataFrame.r
eplace) with CategoricalDtype is deprecated. In a future version, repla
ce will only be used for cases that preserve the categories. To change
the categories, use ser.cat.rename_categories instead.
  colors.replace('No Use','lightgreen', inplace=True)
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/8983811
37.py:6: FutureWarning: The behavior of Series.replace (and DataFrame.r
eplace) with CategoricalDtype is deprecated. In a future version, repla
ce will only be used for cases that preserve the categories. To change
the categories, use ser.cat.rename_categories instead.
  colors.replace('Minimal Use','purple', inplace=True)
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/8983811
37.py:7: FutureWarning: The behavior of Series.replace (and DataFrame.r
eplace) with CategoricalDtype is deprecated. In a future version, repla
ce will only be used for cases that preserve the categories. To change
the categories, use ser.cat.rename_categories instead.
  colors.replace('Moderate Use','lightblue', inplace=True)
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/8983811
37.py:8: FutureWarning: The behavior of Series.replace (and DataFrame.r
eplace) with CategoricalDtype is deprecated. In a future version, repla
ce will only be used for cases that preserve the categories. To change
the categories, use ser.cat.rename_categories instead.
  colors.replace('Primary Pitch','orange', inplace=True)
```



In [469…

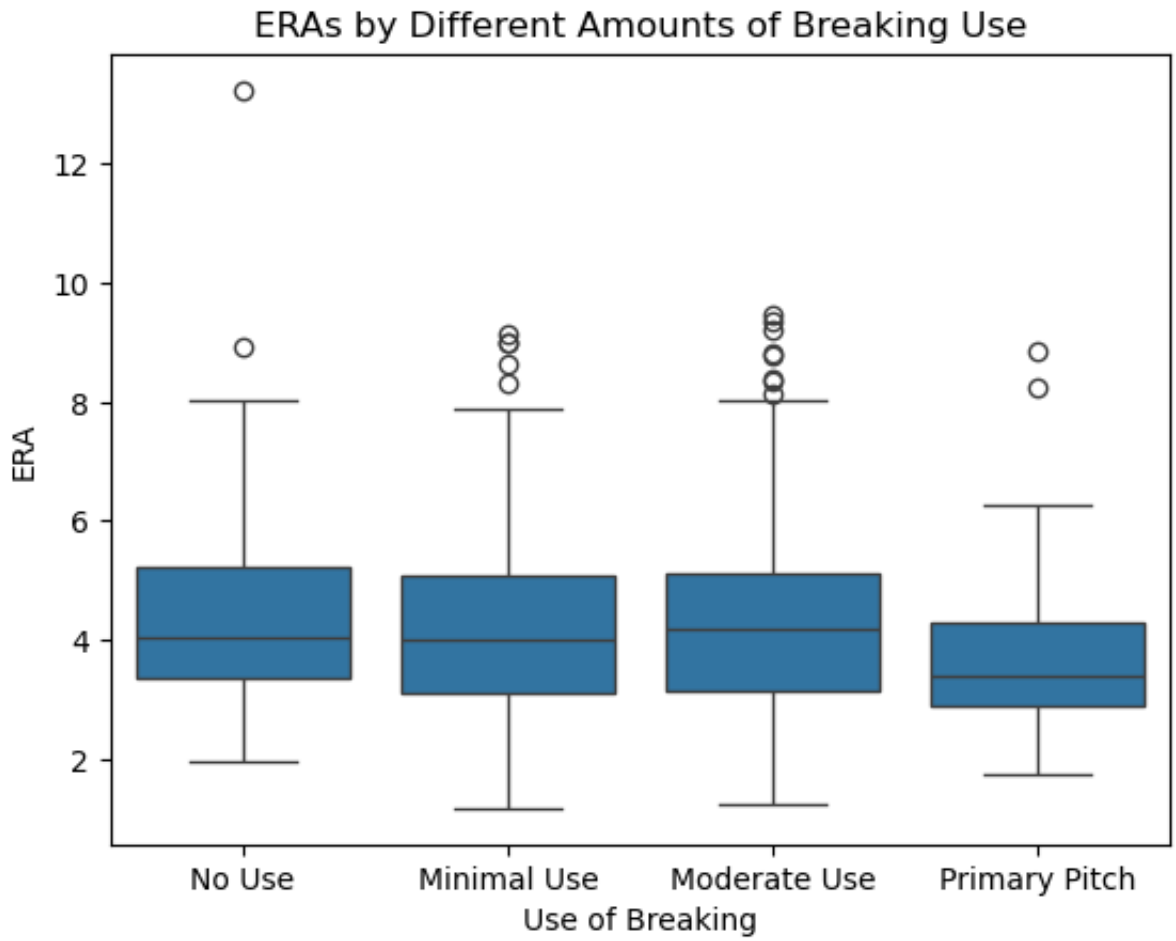
```
#Using boxplots to look at distrubution of ERAs by Use of Fastball - I
#a numerical values and a categorical value, so boxplots make the most

#I am using seaborn for the boxplotsin order to list the categories in
#makes the most logical sense.
#I asked ChatGPT "How would I change the order that categories appear
#in pandas". This showed me how this can be done in seaborn for all of
#boxplots.
import seaborn as sns
sns.boxplot(pitch, x='Use of Fastball', y='ERA', log_scale=False, orde
```

```
plt.title('ERAs by Different Amounts of Fastball Use')
plt.xlabel('Use of Fastball')
plt.ylabel('ERA')

plt.show()

#Analysis: There is not much variation in the mean across fastball usa
#pitchers use fastballs more, there is a more variation in their ERAs.
#Pitchers with really good fastballs will use them more, but as more f
#thrown, there is a higher chance of extra base hits.
```



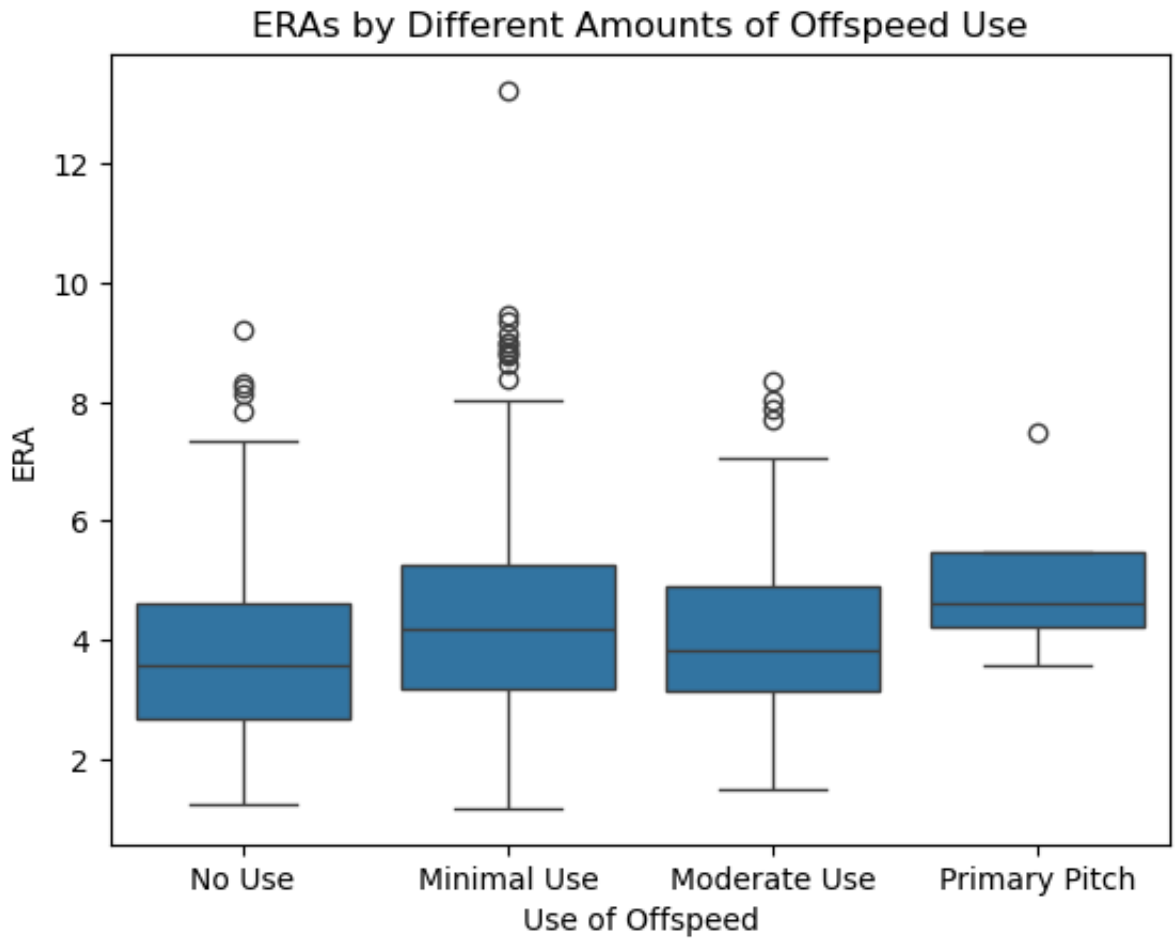
In [470…

```
#Using boxplots to look at distrubution of ERAs by Use of Breaking - I
#a numerical values and a categorical value, so boxplots make the most
sns.boxplot(pitch, x='Use of Breaking', y='ERA', log_scale=False, orde

plt.title('ERAs by Different Amounts of Breaking Use')
plt.xlabel('Use of Breaking')
plt.ylabel('ERA')

plt.show()

#Analysis: Pitchers who use a breaking pitch as their primary pitch ar
#lower ERAs and less variation. This may be due to their breaking pitc
#being better than others, leading to higher usage.
```



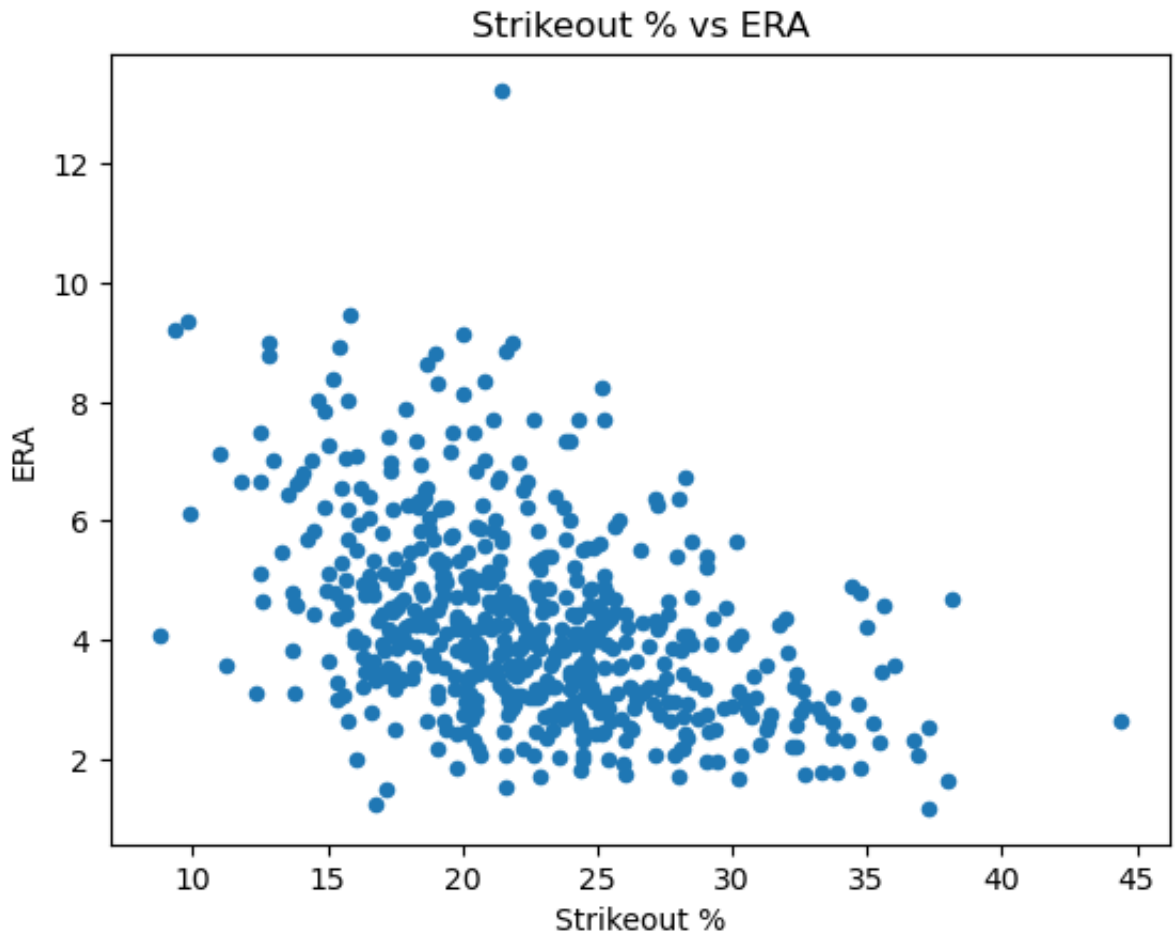
In [471…

```
#Using boxplots to look at distrubution of ERAs by Use of Offspeed - I
#a numerical values and a categorical value, so boxplots make the most
sns.boxplot(pitch, x='Use of Offspeed', y='ERA', log_scale=False, orde

plt.title('ERAs by Different Amounts of Offspeed Use')
plt.xlabel('Use of Offspeed')
plt.ylabel('ERA')

plt.show()

#Analysis: It seems like more offspeed usage is correlated with higher
```



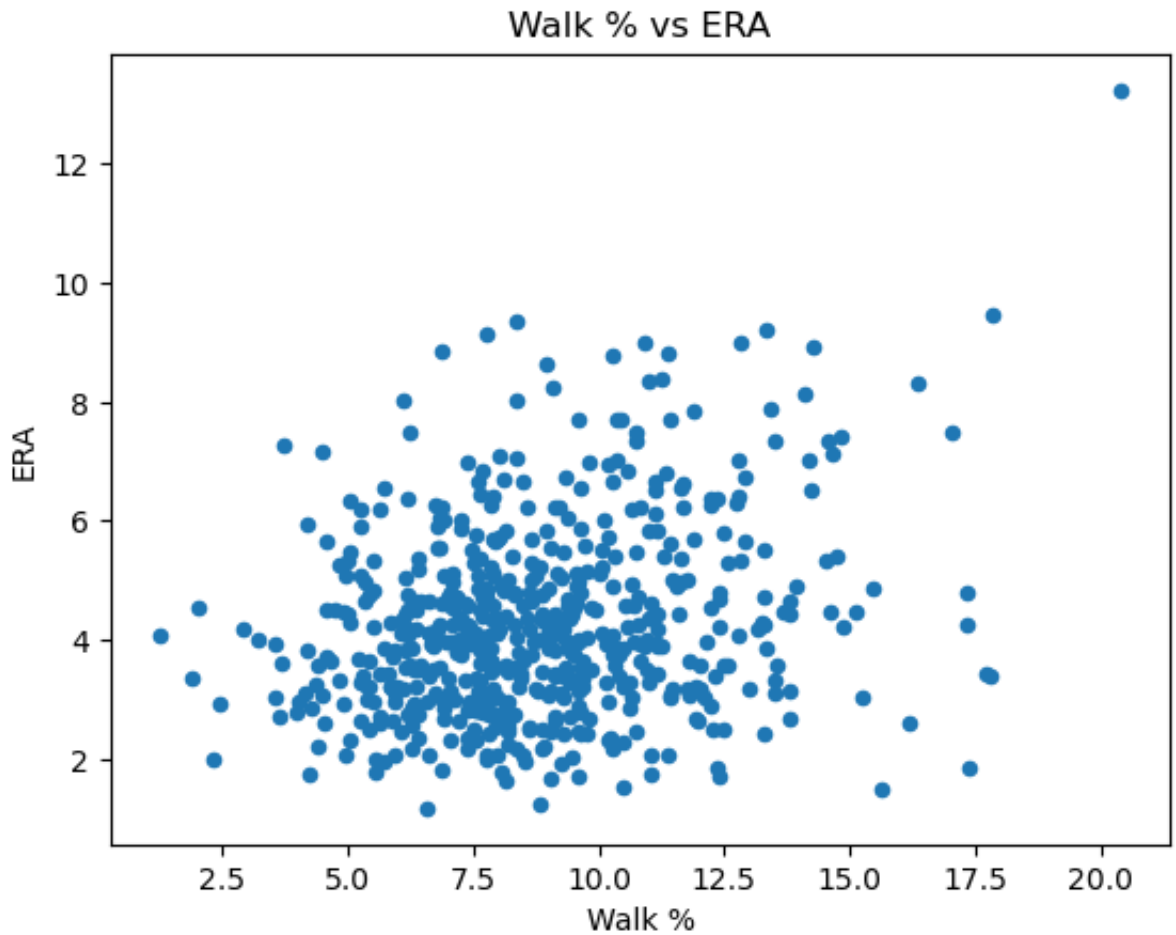
In [472…

```
#Looking at the correlation between Strikeouts and ERA - I am comparin
#values, so a scatterplot makes sense.
#Nothing needs to be color correlated because we are only looking at t
pitch.plot(kind='scatter',
              x = 'Strikeout %',
              y = 'ERA')

plt.title('Strikeout % vs ERA')
plt.xlabel('Strikeout %')
plt.ylabel('ERA')

plt.show()

#Analysis: There is not much correlation between strikeouts and ERA at
#percentages. However, high strikeout percentages seem to correlate wi
```



In [473…

```
#Looking at the correlation between Walks and ERA - I am comparing two
#values, so a scatterplot makes sense.
#Nothing needs to be color correlated because we are only looking at t
pitch.plot(kind='scatter',
              x = 'Walk %',
              y = 'ERA')

plt.title('Walk % vs ERA')
plt.xlabel('Walk %')
plt.ylabel('ERA')

plt.show()

#Analysis: There is some correlation between higher walk rates and hig
```



In [474…

```
#Project Assignment 5 starts below
```

In [475…

```
#Grouping Data - I am grouping the data by pitch type and then looking
#median, and standard deviation for ERA and exit velocity. This will h
#my first research question on pitch type vs success.

#In addition, I am using the grouping to look at the mean, median, and
# of walk and strikeout percentages to answer my second research quest

#Because my third research question involves all numerical data, group
#be used.

grouped_fb = pitch.groupby(pitch['Use of Fastball'])
print(grouped_fb['ERA'].agg(['mean', 'median', 'std']))
print(grouped_fb['Exit Velo AVG'].agg(['mean', 'median', 'std']))
print(grouped_fb['Strikeout %'].agg(['mean', 'median', 'std']))
print(grouped_fb['Walk %'].agg(['mean', 'median', 'std']))

grouped_breaking = pitch.groupby(pitch['Use of Breaking'])
print(grouped_breaking['ERA'].agg(['mean', 'median', 'std']))
print(grouped_breaking['Exit Velo AVG'].agg(['mean', 'median', 'std'])
print(grouped_breaking['Strikeout %'].agg(['mean', 'median', 'std']))
print(grouped_breaking['Walk %'].agg(['mean', 'median', 'std']))
```

```
grouped_offspeed = pitch.groupby(pitch['Use of Offspeed'])
print(grouped_offspeed['ERA'].agg(['mean', 'median', 'std']))
print(grouped_offspeed['Exit Velo AVG'].agg(['mean', 'median', 'std']))
print(grouped_offspeed['Strikeout %'].agg(['mean', 'median', 'std']))
print(grouped_offspeed['Walk %'].agg(['mean', 'median', 'std']))
```

```
                     mean  median       std
Use of Fastball
No Use                NaN     NaN       NaN
Minimal Use      3.440000   3.960  1.328646
Moderate Use     4.179516   3.925  1.542280
Primary Pitch    4.343127   4.120  1.662712
                      mean  median       std
Use of Fastball
No Use                 NaN     NaN       NaN
Minimal Use      88.033333    88.3  0.929157
Moderate Use     89.247849    89.3  1.567515
Primary Pitch    89.294315    89.3  1.596846
                      mean     median       std
Use of Fastball
No Use                 NaN        NaN       NaN
Minimal Use      22.784605  23.701299  3.686170
Moderate Use     22.719233  22.302296  4.995516
Primary Pitch    22.294227  21.604938  5.528333
                      mean     median       std
Use of Fastball
No Use                 NaN        NaN       NaN
Minimal Use      10.528011  10.714286  1.186630
Moderate Use      8.919412   8.926990  2.742507
Primary Pitch     8.741195   8.154506  2.854246
                     mean  median       std
Use of Breaking
No Use           4.429041    4.04  1.751408
Minimal Use      4.261915    4.02  1.579522
Moderate Use     4.328455    4.18  1.627305
Primary Pitch    3.860000    3.40  1.616192
                      mean  median       std
Use of Breaking
No Use           88.894521    89.1  1.775520
Minimal Use      89.257447    89.3  1.462486
Moderate Use     89.445064    89.5  1.617732
Primary Pitch    89.017143    89.1  1.637158
                      mean     median       std
Use of Breaking
No Use           22.669829  23.396226  4.947315
Minimal Use      21.708488  21.241050  5.110422
Moderate Use     22.844567  22.556391  5.770569
Primary Pitch    24.080592  24.104235  4.211844
                     mean    median       std
Use of Breaking
No Use           9.065428  8.547009  3.090135
Minimal Use      8.516557  7.986689  2.839345
Moderate Use     8.957026  8.771930  2.689756
```

```
Primary Pitch    9.236655  9.090909  2.780890
                     mean  median       std
Use of Offspeed
No Use           3.889494    3.58  1.669124
Minimal Use      4.388269    4.17  1.622375
Moderate Use     4.096494    3.83  1.521509
Primary Pitch    5.070000    4.61  1.700294
                      mean  median       std
Use of Offspeed
No Use           88.941772    89.1  1.793908
Minimal Use      89.388221    89.4  1.536160
Moderate Use     89.023377    89.0  1.589347
Primary Pitch    88.600000    88.7  0.886942
                      mean     median       std
Use of Offspeed
No Use           24.426558  24.431818  5.589042
Minimal Use      21.993987  21.438520  5.275459
Moderate Use     22.504807  22.471910  4.771015
Primary Pitch    27.482543  27.601790  9.154387
                      mean     median       std
Use of Offspeed
No Use            9.382657   8.849558  3.406695
Minimal Use       8.643900   8.265433  2.691528
Moderate Use      8.909564   8.658009  2.604363
Primary Pitch    12.577113  11.805243  3.151021
```

```
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/4805218
69.py:11: FutureWarning: The default of observed=False is deprecated an
d will be changed to True in a future version of pandas. Pass observed=
False to retain current behavior or observed=True to adopt the future d
efault and silence this warning.
  grouped_fb = pitch.groupby(pitch['Use of Fastball'])
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/4805218
69.py:17: FutureWarning: The default of observed=False is deprecated an
d will be changed to True in a future version of pandas. Pass observed=
False to retain current behavior or observed=True to adopt the future d
efault and silence this warning.
  grouped_breaking = pitch.groupby(pitch['Use of Breaking'])
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/4805218
69.py:23: FutureWarning: The default of observed=False is deprecated an
d will be changed to True in a future version of pandas. Pass observed=
False to retain current behavior or observed=True to adopt the future d
efault and silence this warning.
  grouped_offspeed = pitch.groupby(pitch['Use of Offspeed'])
```

In [476…

```
#Apply Function and other group operations - Because we are looking mo
#average and spread rather than looking at the outliers or manipulatin
#further, using the apply function will not be beneficial to my projec
```

In [477…

```
#Inputting missing data by group medians - There is no missing data to
#my data. Therefore, this step is not applicable
```

In [478…

```
#Pivot Tables - using to organize data by different combinations of pi
```

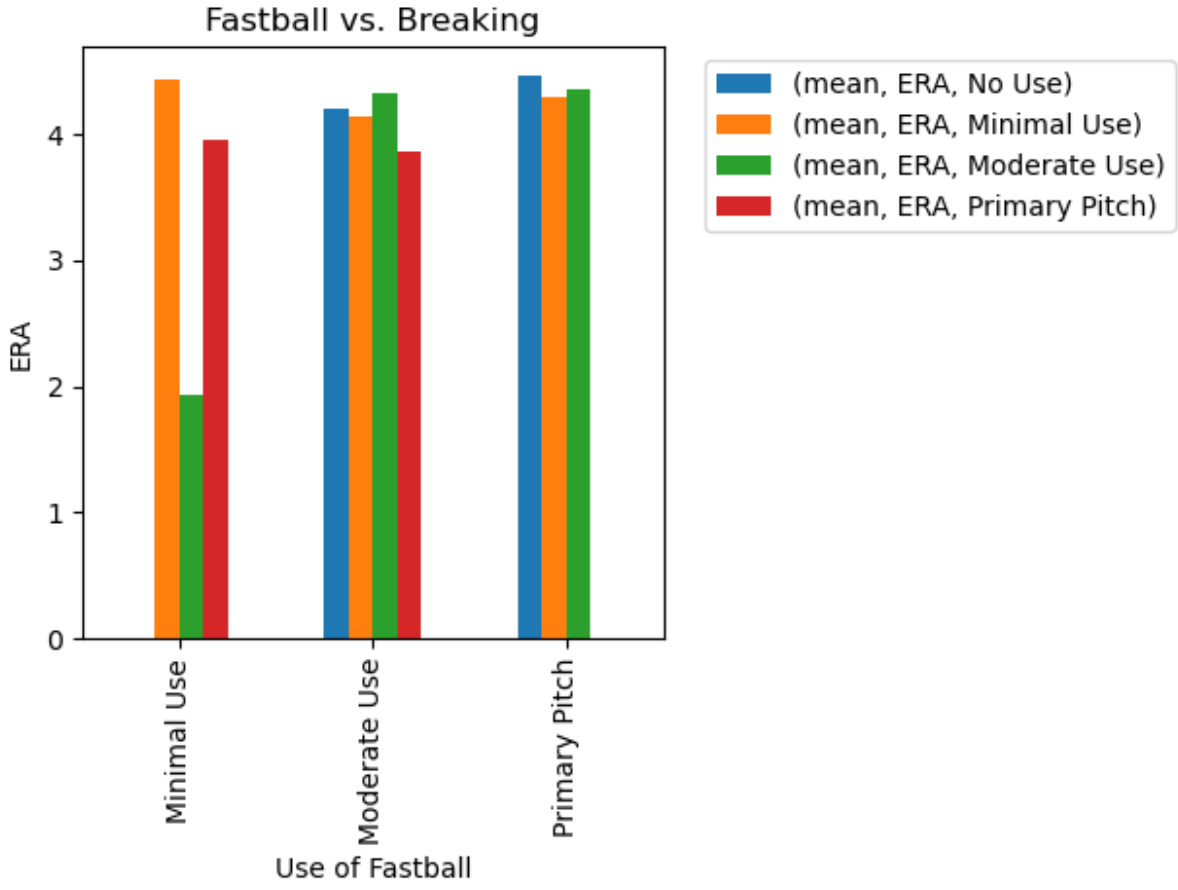
```
#Asked ChatGPT "why are my legends blocking the graph in python?". Thi
#the code for the plt.legend and plt.tight_layout functions used throu
#remainder of this project assignment. I need the legend to show what
#represents, but I did not want it to block any of the bars.

#Fastball vs. Breaking:

fbbreak1 = pitch.pivot_table(index=['Use of Fastball','Use of Breaking
                 values=['ERA'], aggfunc=['mean'])
print(fbbreak1)
fbbreak1 = fbbreak1.unstack()

fbbreak2 = pitch.pivot_table(index=['Use of Fastball','Use of Breaking
                 values=['Exit Velo AVG'], aggfunc=['mean'])
print(fbbreak2)
fbbreak2 = fbbreak2.unstack()

fbbreak3 = pitch.pivot_table(index=['Use of Fastball','Use of Breaking
                 values=['Strikeout %', 'Walk %'], aggfunc=['mean'])
print(fbbreak3)
fbbreak3 = fbbreak3.unstack()

bcfb1 = fbbreak1.plot(kind='bar', title='Fastball vs. Breaking')
plt.ylabel('ERA')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
bcfb2 = fbbreak2.plot(kind='bar', title='Fastball vs. Breaking')
plt.ylabel('Exit Velo AVG')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
bcfb3 = fbbreak3.plot(kind='bar', title='Fastball vs. Breaking')
plt.ylabel('Percentage')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

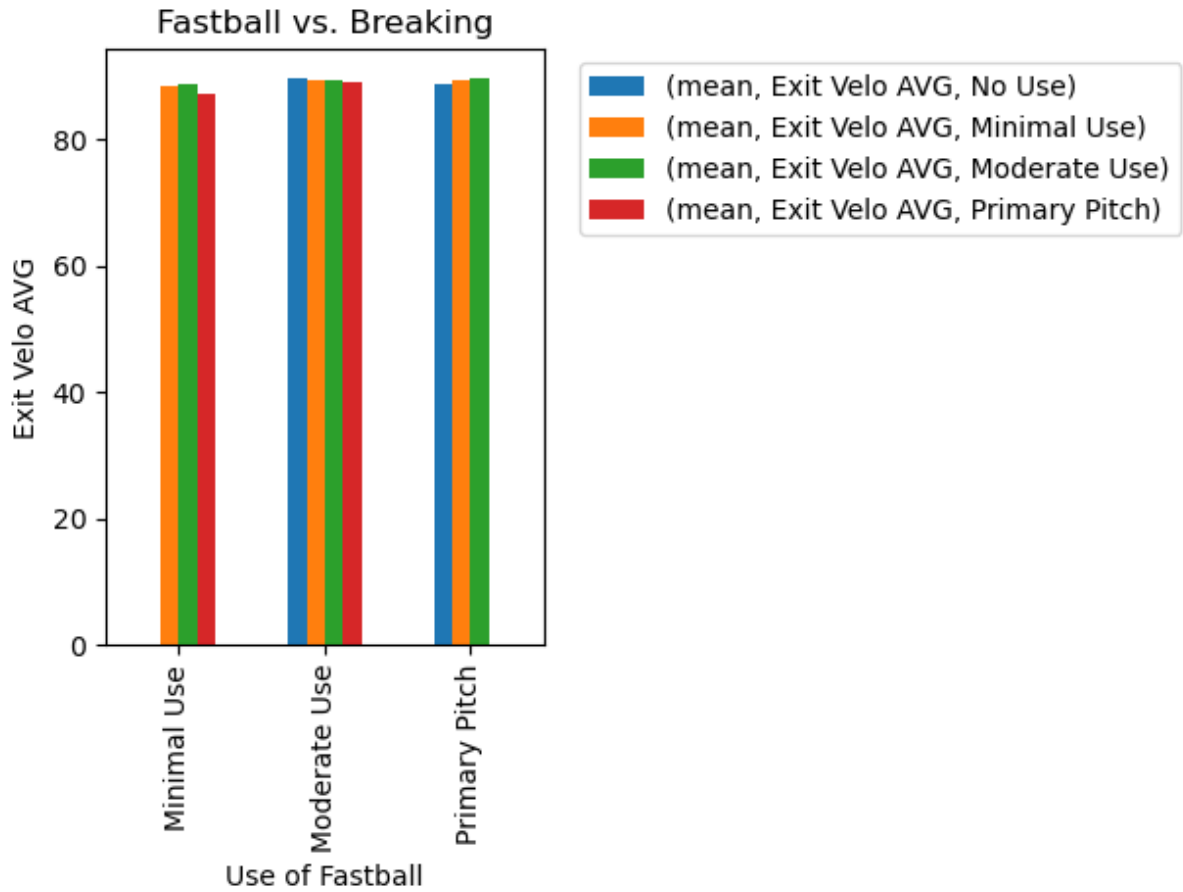
Use of Fastball	Use of Breaking	mean ERA
Minimal Use	Minimal Use	4.430000
	Moderate Use	1.930000
	Primary Pitch	3.960000
Moderate Use	No Use	4.194000
	Minimal Use	4.143137
	Moderate Use	4.318791
	Primary Pitch	3.857059
Primary Pitch	No Use	4.466349
	Minimal Use	4.294098
	Moderate Use	4.351702

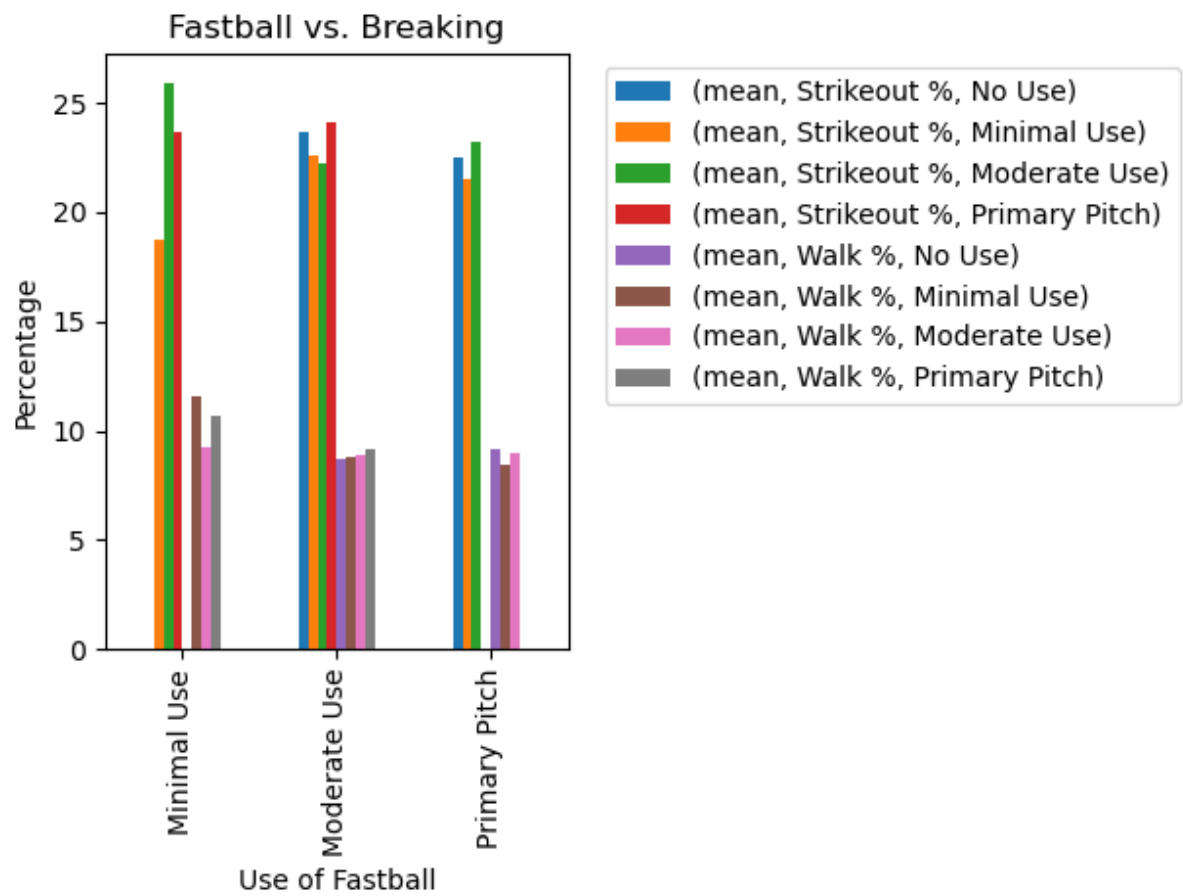
Use of Fastball	Use of Breaking	mean Exit Velo AVG
Minimal Use	Minimal Use	88.300000
	Moderate Use	88.800000
	Primary Pitch	87.000000
Moderate Use	No Use	89.670000
	Minimal Use	89.423529
	Moderate Use	89.167033
	Primary Pitch	89.076471
Primary Pitch	No Use	88.771429
	Minimal Use	89.216393
	Moderate Use	89.629078

Use of Fastball	Use of Breaking	mean Strikeout %	Walk %
Minimal Use	Minimal Use	18.726592	11.610487
	Moderate Use	25.925926	9.259259
	Primary Pitch	23.701299	10.714286
Moderate Use	No Use	23.635648	8.678502
	Minimal Use	22.555733	8.834880
	Moderate Use	22.197353	8.890968
	Primary Pitch	24.091748	9.193195
Primary Pitch	No Use	22.516525	9.126845
	Minimal Use	21.488666	8.410937
	Moderate Use	23.240420	8.997515

```
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/2154775
215.py:10: FutureWarning: The default value of observed=False is deprec
ated and will change to observed=True in a future version of pandas. Sp
ecify observed=False to silence this warning and retain the current beh
avior
  fbbreak1 = pitch.pivot_table(index=['Use of Fastball','Use of Breakin
g'],
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/2154775
215.py:15: FutureWarning: The default value of observed=False is deprec
ated and will change to observed=True in a future version of pandas. Sp
ecify observed=False to silence this warning and retain the current beh
avior
  fbbreak2 = pitch.pivot_table(index=['Use of Fastball','Use of Breakin
g'],
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/2154775
215.py:20: FutureWarning: The default value of observed=False is deprec
ated and will change to observed=True in a future version of pandas. Sp
ecify observed=False to silence this warning and retain the current beh
avior
  fbbreak3 = pitch.pivot_table(index=['Use of Fastball','Use of Breakin
g'],
```







In [479…
```
#Pivot Tables continued
```

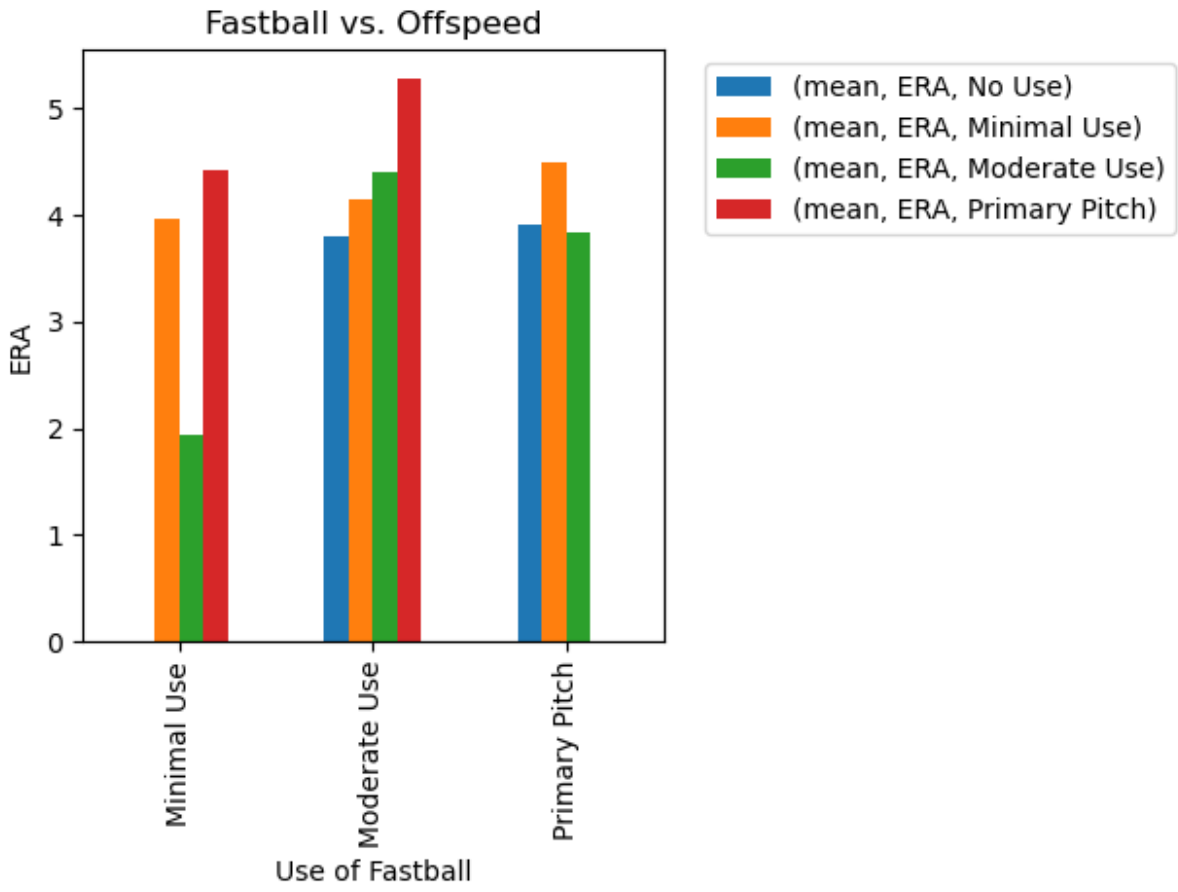
```
#Fastball vs Offspeed:

fboffspeed1 = pitch.pivot_table(index=['Use of Fastball','Use of Offsp
                  values=['ERA'], aggfunc=['mean'])
print(fboffspeed1)
fboffspeed1 = fboffspeed1.unstack()

fboffspeed2 = pitch.pivot_table(index=['Use of Fastball','Use of Offsp
                  values=['Exit Velo AVG'], aggfunc=['mean'])
print(fboffspeed2)
fboffspeed2 = fboffspeed2.unstack()

fboffspeed3 = pitch.pivot_table(index=['Use of Fastball','Use of Offsp
                  values=['Strikeout %', 'Walk %'], aggfunc=['mean'])
print(fboffspeed3)
fboffspeed3 = fboffspeed3.unstack()

bcfo1 = fboffspeed1.plot(kind='bar', title='Fastball vs. Offspeed')
plt.ylabel('ERA')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
bcfo2 = fboffspeed2.plot(kind='bar', title='Fastball vs. Offspeed')
plt.ylabel('Exit Velo AVG')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
bcfo3 = fboffspeed3.plot(kind='bar', title='Fastball vs. Offspeed')
plt.ylabel('Percentage')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

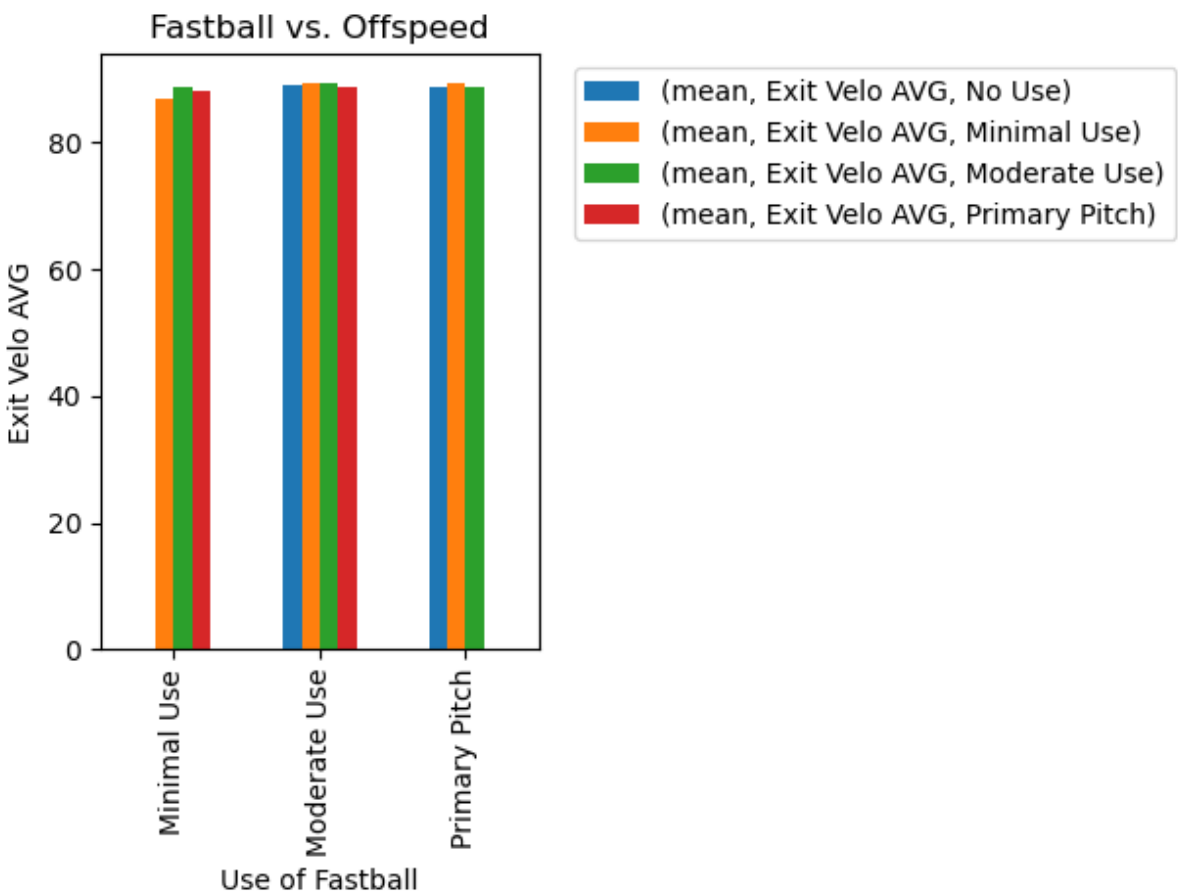
```
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/1606960
162.py:4: FutureWarning: The default value of observed=False is depreca
ted and will change to observed=True in a future version of pandas. Spe
cify observed=False to silence this warning and retain the current beha
vior
  fboffspeed1 = pitch.pivot_table(index=['Use of Fastball','Use of Offs
peed'],
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/1606960
162.py:9: FutureWarning: The default value of observed=False is depreca
ted and will change to observed=True in a future version of pandas. Spe
cify observed=False to silence this warning and retain the current beha
vior
  fboffspeed2 = pitch.pivot_table(index=['Use of Fastball','Use of Offs
peed'],
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/1606960
162.py:14: FutureWarning: The default value of observed=False is deprec
ated and will change to observed=True in a future version of pandas. Sp
ecify observed=False to silence this warning and retain the current beh
avior
  fboffspeed3 = pitch.pivot_table(index=['Use of Fastball','Use of Offs
peed'],
```

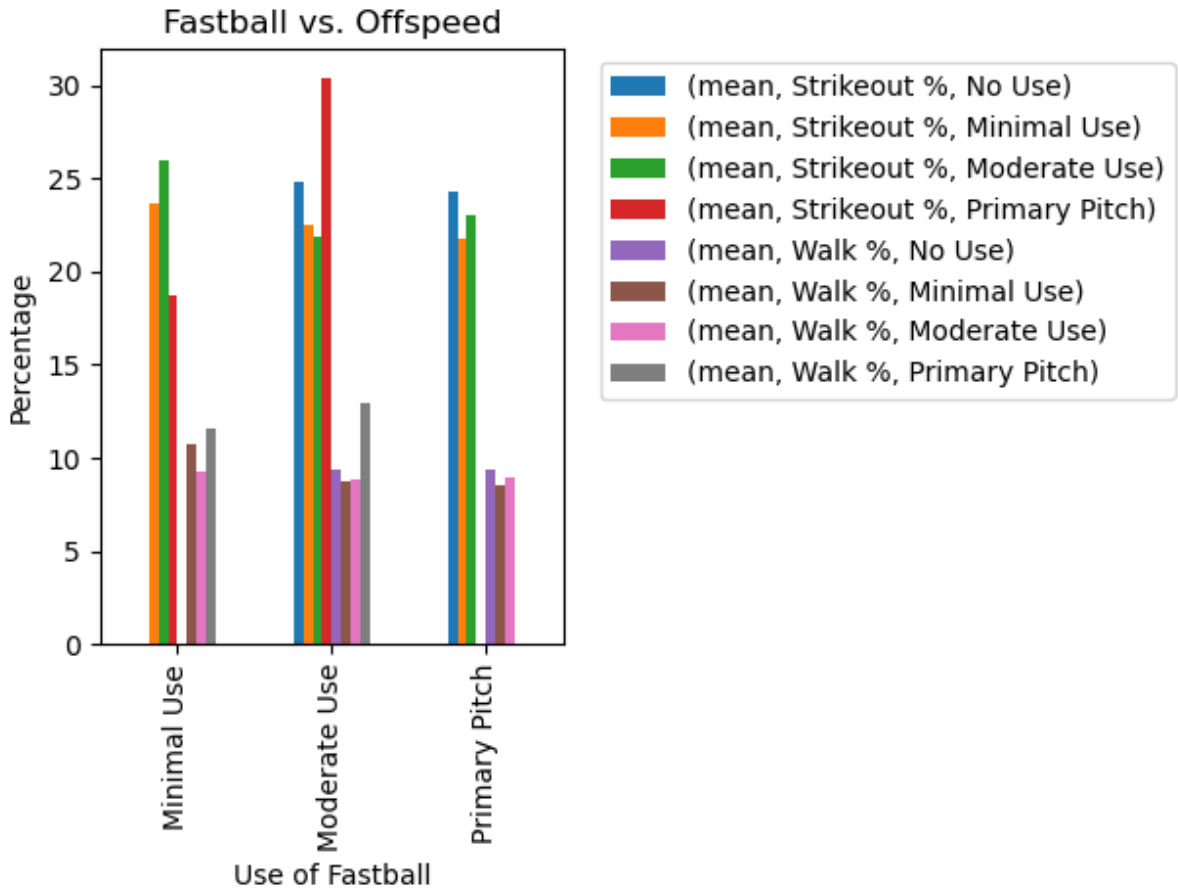
Use of Fastball	Use of Offspeed	mean ERA
Minimal Use	Minimal Use	3.960000
	Moderate Use	1.930000
	Primary Pitch	4.430000
Moderate Use	No Use	3.799444
	Minimal Use	4.139127
	Moderate Use	4.400513
	Primary Pitch	5.283333
Primary Pitch	No Use	3.916066
	Minimal Use	4.498374
	Moderate Use	3.834595

Use of Fastball	Use of Offspeed	mean Exit Velo AVG
Minimal Use	Minimal Use	87.000000
	Moderate Use	88.800000
	Primary Pitch	88.300000
Moderate Use	No Use	89.205556
	Minimal Use	89.252381
	Moderate Use	89.294872
	Primary Pitch	88.700000
Primary Pitch	No Use	88.863934
	Minimal Use	89.455709
	Moderate Use	88.743243

Use of Fastball	Use of Offspeed	mean Strikeout %	Walk %
Minimal Use	Minimal Use	23.701299	10.714286
	Moderate Use	25.925926	9.259259
	Primary Pitch	18.726592	11.610487
Moderate Use	No Use	24.802315	9.364662
	Minimal Use	22.498661	8.784600
	Moderate Use	21.879510	8.843311
	Primary Pitch	30.401193	12.899321
Primary Pitch	No Use	24.315679	9.387967
	Minimal Use	21.768049	8.575392
	Moderate Use	23.071442	8.969947


Fastball vs. Offspeed
ERA
0
1
2
3
4
5
Minimal Use
Moderate Use
Primary Pitch
Use of Fastball
(mean, ERA, No Use)
(mean, ERA, Minimal Use)
(mean, ERA, Moderate Use)
(mean, ERA, Primary Pitch)


Fastball vs. Offspeed
Exit Velo AVG
0
20
40
60
80
Minimal Use
Moderate Use
Primary Pitch
Use of Fastball
(mean, Exit Velo AVG, No Use)
(mean, Exit Velo AVG, Minimal Use)
(mean, Exit Velo AVG, Moderate Use)
(mean, Exit Velo AVG, Primary Pitch)



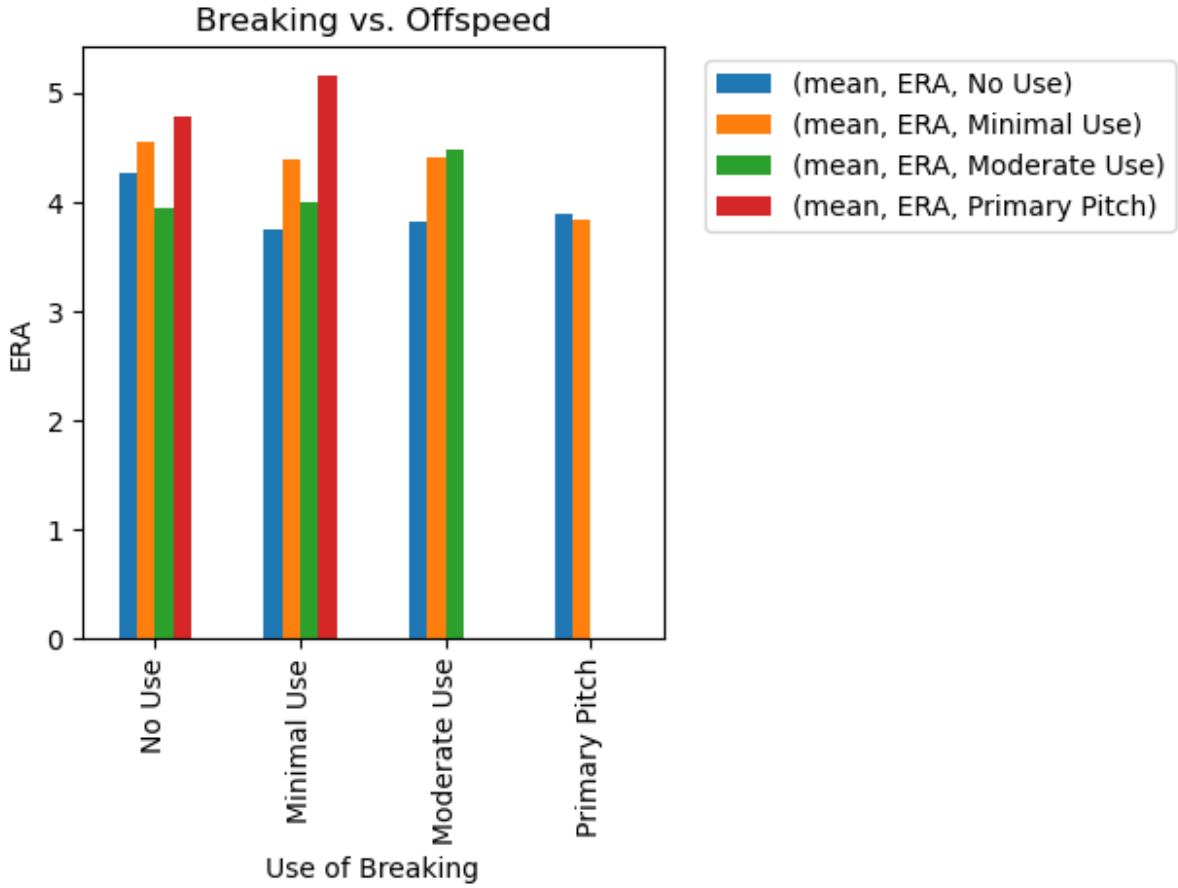
In [480…

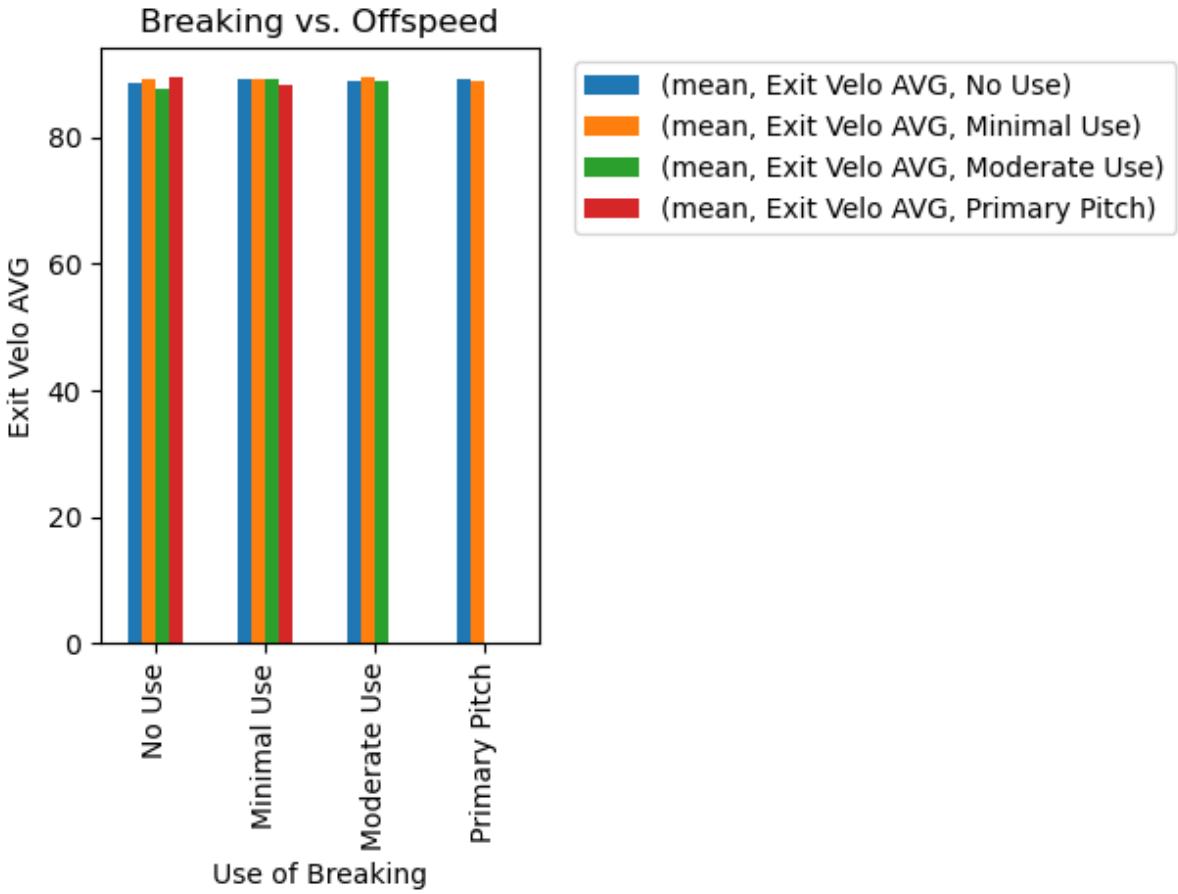
```
breakoffspeed1 = pitch.pivot_table(index=['Use of Breaking','Use of Of
                 values=['ERA'], aggfunc=['mean'])
print(breakoffspeed1)
breakoffspeed1 = breakoffspeed1.unstack()

breakoffspeed2 = pitch.pivot_table(index=['Use of Breaking','Use of Of
                 values=['Exit Velo AVG'], aggfunc=['mean'])
print(breakoffspeed2)
breakoffspeed2 = breakoffspeed2.unstack()

breakoffspeed3 = pitch.pivot_table(index=['Use of Breaking','Use of Of
                 values=['Strikeout %', 'Walk %'], aggfunc=['mean'])
print(breakoffspeed3)
breakoffspeed3 = breakoffspeed3.unstack()

bcbo1 = breakoffspeed1.plot(kind='bar', title='Breaking vs. Offspeed')
plt.ylabel('ERA')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
bcbo2 = breakoffspeed2.plot(kind='bar', title='Breaking vs. Offspeed')
plt.ylabel('Exit Velo AVG')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
bcbo3 = breakoffspeed3.plot(kind='bar', title='Breaking vs. Offspeed')
```

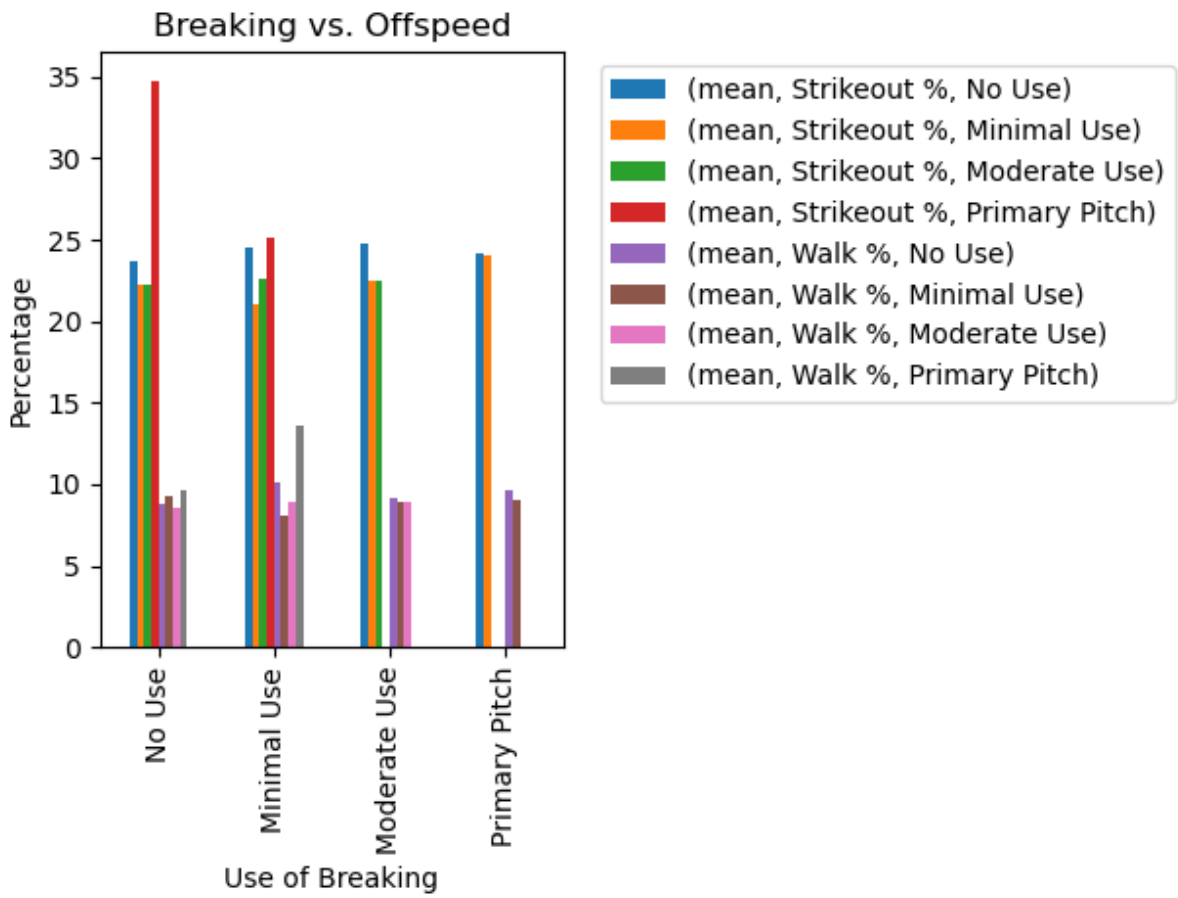
```
plt.ylabel('Percentage')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

```
                                    mean
                                     ERA
Use of Breaking Use of Offspeed
No Use          No Use           4.264615
                Minimal Use      4.549800
                Moderate Use     3.955556
                Primary Pitch    4.790000
Minimal Use     No Use           3.748889
                Minimal Use      4.385309
                Moderate Use     4.003077
                Primary Pitch    5.163333
Moderate Use    No Use           3.827027
                Minimal Use      4.418111
                Moderate Use     4.479375
Primary Pitch   No Use           3.886364
                Minimal Use      3.847917
                                         mean
                                Exit Velo AVG
Use of Breaking Use of Offspeed
No Use          No Use              88.492308
                Minimal Use         89.204000
                Moderate Use        87.688889
                Primary Pitch       89.500000
Minimal Use     No Use              89.233333
                Minimal Use         89.269753
                Moderate Use        89.282692
                Primary Pitch       88.300000
Moderate Use    No Use              88.859459
                Minimal Use         89.611111
                Moderate Use        88.931250
Primary Pitch   No Use              89.272727
                Minimal Use         88.900000
                                       mean
                                Strikeout %     Walk %
Use of Breaking Use of Offspeed
No Use          No Use            23.665372   8.793244
                Minimal Use       22.243721   9.220834
                Moderate Use      22.256960   8.529985
                Primary Pitch     34.749035   9.652510
Minimal Use     No Use            24.498064  10.135761
                Minimal Use       21.063980   8.098807
                Moderate Use      22.557379   8.967007
                Primary Pitch     25.060379  13.551980
Moderate Use    No Use            24.733961   9.154438
                Minimal Use       22.489188   8.918281
                Moderate Use      22.473363   8.936389
Primary Pitch   No Use            24.175141   9.614529
                Minimal Use       24.037257   9.063463
```

```
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/3353657
881.py:1: FutureWarning: The default value of observed=False is depreca
ted and will change to observed=True in a future version of pandas. Spe
cify observed=False to silence this warning and retain the current beha
vior
  breakoffspeed1 = pitch.pivot_table(index=['Use of Breaking','Use of O
ffspeed'],
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/3353657
881.py:6: FutureWarning: The default value of observed=False is depreca
ted and will change to observed=True in a future version of pandas. Spe
cify observed=False to silence this warning and retain the current beha
vior
  breakoffspeed2 = pitch.pivot_table(index=['Use of Breaking','Use of O
ffspeed'],
/var/folders/tt/sk4dmlrj0r5gpbz1y0njnvdm0000gn/T/ipykernel_1251/3353657
881.py:11: FutureWarning: The default value of observed=False is deprec
ated and will change to observed=True in a future version of pandas. Sp
ecify observed=False to silence this warning and retain the current beh
avior
  breakoffspeed3 = pitch.pivot_table(index=['Use of Breaking','Use of O
ffspeed'],
```




Breaking vs. Offspeed
Exit Velo AVG
0
20
40
60
80
No Use
Minimal Use
Moderate Use
Primary Pitch
Use of Breaking
(mean, Exit Velo AVG, No Use)
(mean, Exit Velo AVG, Minimal Use)
(mean, Exit Velo AVG, Moderate Use)
(mean, Exit Velo AVG, Primary Pitch)



In [481…

```
#Cross-tabulation - using this to see the amounts of each type of two-
#that was put together in the pivots above. This will help me determin
#are enough data points in the population for each combination for it
#looking into. I don't think creating a bar chart for any of these wil
#because the totals are not being used to actually show anything that
```

```
#in my final conclusions.

#Since there isn't any other categorical data to look at, I don't thin
#crosstabs for anything else is necessary.

#All Minimal Fastball combinations and all Offspeed as a primary pitch
#do not have a large enough sample size.
#The combination of no breaking use and moderate offspeed use might be
#to analyze as well.

print(pd.crosstab(pitch['Use of Fastball'], pitch['Use of Breaking']))
print(pd.crosstab(pitch['Use of Fastball'], pitch['Use of Offspeed']))
print(pd.crosstab(pitch['Use of Breaking'], pitch['Use of Offspeed']))
```

```
Use of Breaking  No Use  Minimal Use  Moderate Use  Primary Pitch
Use of Fastball
Minimal Use           0            1             1              1
Moderate Use         10           51            91             34
Primary Pitch        63          183           141              0
Use of Offspeed  No Use  Minimal Use  Moderate Use  Primary Pitch
Use of Fastball
Minimal Use           0            1             1              1
Moderate Use         18          126            39              3
Primary Pitch        61          289            37              0
Use of Offspeed  No Use  Minimal Use  Moderate Use  Primary Pitch
Use of Breaking
No Use               13           50             9              1
Minimal Use          18          162            52              3
Moderate Use         37          180            16              0
Primary Pitch        11           24             0              0
```

In [482…

```
#Due 3/15/26 - Project Assignment 6:
import statsmodels.api as sm
```

In [483…

```
#Research Question 1 requires ANOVA.
#Research Questions 2 and 3 require regression.
#I assigned my categorical columns for pitch usage numerical values in
#tests. The results below seemed a lot more efficient than getting dum
#for each column.

#Asked ChatGPT: "how to assign a category to a numerical value in pyth
#and clarified with "they are currently categories, I want to assign
#them numerical values".

pitch['FB Use'] = pitch['Use of Fastball'].map({
    'No Use': 0,
    'Minimal Use': 1,
    'Moderate Use': 2,
    'Primary Pitch': 3
 })

pitch['Breaking Use'] = pitch['Use of Breaking'].map({
    'No Use': 0,
```

```
    'Minimal Use': 1,
    'Moderate Use': 2,
    'Primary Pitch': 3
 })

pitch['Offspeed Use'] = pitch['Use of Offspeed'].map({
    'No Use': 0,
    'Minimal Use': 1,
    'Moderate Use': 2,
    'Primary Pitch': 3
 })

pitchuse = pitch[[ 'FB Use', 'Breaking Use', 'Offspeed Use']]
pitchuse
```

Out[483…

	FB Use	Breaking Use	Offspeed Use
0	2	3	1
1	3	1	2
2	3	1	1
3	3	1	1
4	2	3	0
...	...	...	...
571	2	2	1
572	3	1	1
573	2	2	1
574	3	2	1
575	3	1	0

576 rows × 3 columns

In [484…

```
#Research Question 2 - Using regression template from Lab 8 to demonst
#relationship between pitch type and Strikeout %.

X = pitchuse
y = pitch['Strikeout %']

X = sm.add_constant(X)
model1 = sm.OLS(y, X).fit()

print(model1.rsquared_adj)
model1.summary()

#Not statistically significant
```

```
0.004700856877078552
```

Out [484…

OLS Regression Results

Dep. Variable:	Strikeout %	**R-squared:**	0.010
Model:	OLS	**Adj. R-squared:**	0.005
Method:	Least Squares	**F-statistic:**	1.905
Date:	Sat, 14 Mar 2026	**Prob (F-statistic):**	0.128
Time:	09:45:32	**Log-Likelihood:**	-1780.1
No. Observations:	576	**AIC:**	3568.
Df Residuals:	572	**BIC:**	3586.
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	23.3222	1.814	12.859	0.000	19.760	26.885
FB Use	-0.3157	0.517	-0.611	0.541	-1.330	0.699
Breaking Use	0.4031	0.316	1.276	0.203	-0.218	1.024
Offspeed Use	-0.6041	0.432	-1.397	0.163	-1.453	0.245

Omnibus:	27.828	**Durbin-Watson:**	2.034
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	31.252
Skew:	0.511	**Prob(JB):**	1.64e-07
Kurtosis:	3.507	**Cond. No.**	28.6

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [485…

```
#Research Question 2 – Using regression template from Lab 8 to demonst
#and Walk %.
X = pitchuse
y = pitch['Walk %']

X = sm.add_constant(X)
model2 = sm.OLS(y, X).fit()

print(model2.rsquared_adj)
model2.summary()
```

```
#Not statistically significant
```

```
-0.0029857967829183263
```

Out [485…

OLS Regression Results

Dep. Variable:	Walk %	**R-squared:**	0.002
Model:	OLS	**Adj. R-squared:**	-0.003
Method:	Least Squares	**F-statistic:**	0.4294
Date:	Sat, 14 Mar 2026	**Prob (F-statistic):**	0.732
Time:	09:45:32	**Log-Likelihood:**	-1412.0
No. Observations:	576	**AIC:**	2832.
Df Residuals:	572	**BIC:**	2849.
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	9.2909	0.957	9.705	0.000	7.411	11.171
FB Use	-0.1996	0.273	-0.732	0.464	-0.735	0.336
Breaking Use	0.0782	0.167	0.469	0.639	-0.249	0.406
Offspeed Use	-0.0594	0.228	-0.260	0.795	-0.508	0.389

Omnibus:	38.646	**Durbin-Watson:**	2.116
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	47.870
Skew:	0.588	**Prob(JB):**	4.03e-11
Kurtosis:	3.782	**Cond. No.**	28.6

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [486…

```
# I took a picture of the output and asked Copilot to "help me to inte
#results of the regression I ran in python". I wanted more information
#of the columns headers represent in case there was any statistical si
#the variables.

#Per the results showing R-squared close to zero and each individual
#pitch type's p-value being much greater than .05 for both tests, ther
```

```
#statistically significant relationship between the usage of any indiv
#and a pitcher's strikeout or walk rate.
```

In [487…

```
#Research Question 1 - running ANOVA Tests to compare ERA among pitch

#Asked Copilot "I want to run an anova test to test the difference in
#three different dataframes. How should I approach this?".

g1 = pitch[pitch['Use of Fastball'] == 'Primary Pitch']
g2 = pitch[pitch['Use of Breaking'] == 'Primary Pitch']
g3 = pitch[pitch['Use of Offspeed'] == 'Primary Pitch']

g1 = g1['ERA']
g2 = g2['ERA']
g3 = g3['ERA']

from scipy.stats import f_oneway

F, p = f_oneway(g1, g2, g3)

print("F-statistic:", F)
print("p-value:", p)

#not statistical significance between which pitch is a primary pitch a
```

```
F-statistic: 1.7838134159620935
p-value: 0.1692576766788469
```

In [488…

```
#Research Question 1 continued:
h1 = pitch[pitch['Use of Fastball'] == 'Moderate Use']
h2 = pitch[pitch['Use of Breaking'] == 'Moderate Use']
h3 = pitch[pitch['Use of Offspeed'] == 'Moderate Use']

h1 = h1['ERA']
h2 = h2['ERA']
h3 = h3['ERA']

F, p = f_oneway(h1, h2, h3)

print("F-statistic:", F)
print("p-value:", p)

#not statistical significance between moderate pitch usages and ERA.
```

```
F-statistic: 0.8181249040032579
p-value: 0.44185644419044967
```

In [489…

```
#Research Question 1 continued:
i1 = pitch[pitch['Use of Fastball'] == 'Minimal Use']
i2 = pitch[pitch['Use of Breaking'] == 'Minimal Use']
i3 = pitch[pitch['Use of Offspeed'] == 'Minimal Use']

i1 = i1['ERA']
```

```
i2 = i2['ERA']
i3 = i3['ERA']

F, p = f_oneway(i1, i2, i3)

print("F-statistic:", F)
print("p-value:", p)

#not statistical significance between minimal pitch usages and ERA.
```

```
F-statistic: 0.9361333949299877
p-value: 0.3926683890052365
```

In [490…

```
#Research Question 1 continued:
#No 'Use of Fastball' equals 'No Use'
j2 = pitch[pitch['Use of Breaking'] == 'No Use']
j3 = pitch[pitch['Use of Offspeed'] == 'No Use']

j2 = j2['ERA']
j3 = j3['ERA']

F, p = f_oneway(j2, j3)

print("F-statistic:", F)
print("p-value:", p)

print(j2.mean())
print(j3.mean())

#Looks like we finally found something somewhat statistically signific
#(p-value = 0.05). There is a difference in ERAs between pitchers who
#not use Breaking pitches and pitchers who do not use offspeed pitches
#either that some use of breaking pitches leads to lower ERAs or less
#offspeed may lead to lower ERAs. Let's test further.

#Next, I will prepare tests to compare among each fb group against eac
#breaking group against each other, and each offspeed group against ea
#FB
F, p = f_oneway(g1, h1, i1)

print("F-statistic:", F)
print("p-value:", p)

#Breaking
F, p = f_oneway(g2, h2, i2, j2)

print("F-statistic:", F)
print("p-value:", p)

F, p = f_oneway(g3, h3, i3, j3)

#Offspeed
print("F-statistic:", F)
```

```
print("p-value:", p)

print(g3.mean())
print(h3.mean())
print(i3.mean())
print(j3.mean())

#It appears that there is statistcal significance to the difference in
#the different groups of offspeed usage. No offspeed usage provides th
#and using offspeed as a primary pitch leads to the highest ERA. There
#may be an advantage to limiting offspeed use.
```

```
F-statistic: 3.781143321745996
p-value: 0.053704141824684776
4.429041095890412
3.889493670886074
F-statistic: 1.0466564033062487
p-value: 0.3517800355802362
F-statistic: 1.0623736480346742
p-value: 0.3645443770575376
F-statistic: 2.806574095948356
p-value: 0.03902946966061882
5.07
4.0964935064935055
4.388269230769231
3.889493670886074
```

In [491…

```
#Research Question 3 - I am running regressions to look at how strikeo
#rates affect a pitcher's overall performance.

kbb = pitch[['Strikeout %', 'Walk %']]
X = kbb
y = pitch['ERA']

X = sm.add_constant(X)
model5 = sm.OLS(y, X).fit()

print(model5.rsquared_adj)
model5.summary()

#There is a storng statistical significance between a pitchers strikeo
#with their ERAs. About 25.7% of variance in ERAs is explained by thes
```

```
0.25447457100084714
```

Out [491…

OLS Regression Results

Dep. Variable:	ERA	**R-squared:**	0.257
Model:	OLS	**Adj. R-squared:**	0.254
Method:	Least Squares	**F-statistic:**	99.13
Date:	Sat, 14 Mar 2026	**Prob (F-statistic):**	1.06e-37
Time:	09:45:32	**Log-Likelihood:**	-1010.5
No. Observations:	576	**AIC:**	2027.
Df Residuals:	573	**BIC:**	2040.
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	5.9715	0.306	19.510	0.000	5.370	6.573
Strikeout %	-0.1347	0.011	-12.318	0.000	-0.156	-0.113
Walk %	0.1517	0.021	7.293	0.000	0.111	0.193

Omnibus:	52.039	**Durbin-Watson:**	2.035
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	81.965
Skew:	0.626	**Prob(JB):**	1.59e-18
Kurtosis:	4.360	**Cond. No.**	129.

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [492…

```
#Research Question 3 continued:

X = kbb
y = pitch['Exit Velo AVG']

X = sm.add_constant(X)
model6 = sm.OLS(y, X).fit()

print(model6.rsquared_adj)
model6.summary()

#There is not a statistically significant relationship between walk ra
#velocities of opposing hitters, but there is a statistically signific
```

```
#between strikeout rates and exit velocities of opposing hitters.
```

```
0.015353721103382512
```

Out[492…

OLS Regression Results

Dep. Variable:	Exit Velo AVG	**R-squared:**	0.019
Model:	OLS	**Adj. R-squared:**	0.015
Method:	Least Squares	**F-statistic:**	5.483
Date:	Sat, 14 Mar 2026	**Prob (F-statistic):**	0.00438
Time:	09:45:32	**Log-Likelihood:**	-1076.8
No. Observations:	576	**AIC:**	2160.
Df Residuals:	573	**BIC:**	2173.
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	90.1079	0.343	262.370	0.000	89.433	90.782
Strikeout %	-0.0405	0.012	-3.303	0.001	-0.065	-0.016
Walk %	0.0084	0.023	0.361	0.718	-0.037	0.054

Omnibus:	8.045	**Durbin-Watson:**	2.164
Prob(Omnibus):	0.018	**Jarque-Bera (JB):**	7.984
Skew:	-0.284	**Prob(JB):**	0.0185
Kurtosis:	3.105	**Cond. No.**	129.

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.